
PersonalOS

Come costruirti il sistema che tiene insieme la tua vita, parlando a un agente

Una guida da trascinare dentro Claude Code: lui la legge tutta e ti accompagna nella costruzione della tua dashboard personale — task, calendario, finanze, abitudini e una memoria che impara chi sei. Tu decidi, lui scrive.

Una guida di Giuseppe Castagna · v1.0 — agosto 2026 ·
youtube.com/@gcastagna

L Percorso Locale — 30-60 minuti · **C** Percorso Completo —
10-15 ore

Indice

Cos'è questo documento

Come si usa

Scegli il percorso — con le avvertenze e cosa ti serve prima di iniziare

Parte 1 · Le scelte di stack

Parte 2 · Disegna prima di costruire

Parte 3 · Le fondamenta

Parte 4 · La cattura

Parte 5 · Le schede

Parte 6 · La memoria

Parte 7 · Metterlo online

Parte 8 · Quando si rompe

Parte 9 · Dove portarlo dopo

Appendice A · Tutti i prompt in sequenza

Appendice B · Checklist variabili d'ambiente

Prima di chiudere

Cos'è questo documento

Non è un tutorial da leggere. È un documento da **dare in pasto a Claude Code**.

Lo apri, lo trascini dentro una chat di Claude Code, e lui lo legge tutto. Da quel momento sa cosa stai costruendo, in che ordine, e con quali scelte. Tu non scrivi codice: decidi. Lui scrive.

Alla fine avrai una dashboard che tiene insieme task, calendario, finanze, abitudini e una memoria che impara chi sei. Le parli — a voce o scrivendo — e lei archivia da sola nel posto giusto. E quando le fai una domanda, ti risponde citando da dove ha preso le informazioni.

Come si usa

1. Apri Claude Code in una cartella vuota
2. Trascina dentro questo PDF
3. Scrivi: *"Ho letto la guida. Partiamo."*
4. Lui ti chiede quale percorso vuoi, e si comincia

La prima volta segui l'ordine. Dopo, questo documento diventa un riferimento da consultare: vai alla parte che ti serve.

Una cosa chiara, prima di partire. Questa è una guida, non un servizio: viene distribuita così com'è, a scopo informativo, senza garanzie di nessun tipo. Quello che costruisci seguendola — i comandi che esegui, gli account che apri, le chiavi che generi, i costi che ne derivano, i dati che ci metti dentro — è opera tua e resta sotto la tua responsabilità. I servizi citati sono di terzi: pannelli, piani e prezzi cambiano senza che questa guida se ne accorga. Verifica sempre sulle fonti ufficiali prima di decidere.

Scegli il percorso

Prima di scrivere una riga, c'è una decisione da prendere. È l'unica importante di tutto il documento, perché cambia il resto.

	Percorso Locale	Percorso Completo
Tempo	30-60 minuti	10-15 ore
Account necessari	solo Claude	+ OpenAI, Supabase, Vercel, Telegram
Dove stanno i dati	un file sul tuo computer	un database vero
Da dove catturi	dalla dashboard, anche a voce	anche da Telegram, ovunque tu sia
La memoria	tutto il contesto passa al modello	ricerca per somiglianza
Dove vive	sulla tua macchina, quando è accesa	online, sempre
Si sveglia da solo	no	sì: briefing la mattina
Costo mensile	quasi zero	i piani gratuiti bastano; paghi il consumo delle API

Quale scegliere

Prendi il Locale se è la prima volta che costruisci qualcosa del genere, se vuoi capire come funziona prima di impegnarti, o se semplicemente vuoi qualcosa che funzioni stasera. Fa tutto quello che serve tranne una cosa: non puoi catturare un pensiero mentre sei in fila alla cassa.

Prendi il Completo se quella cosa lì è esattamente il motivo per cui sei qui. Se il valore, per te, è mandare una nota vocale da ovunque e ritrovarla già in ordine — allora ti serve qualcosa che stia sveglio anche quando il tuo computer dorme, e quello costa tempo e qualche account.

Si sale, non si scende. Puoi partire dal Locale e passare al Completo più avanti: i dati si portano dietro, e la procedura per travasarli è scritta — è la voce 7 della Parte 9. Il contrario è più faticoso. Nel dubbio, parti dal Locale.

Le avvertenze — leggi adesso, non a metà strada

Cosa devi già saper fare. Onestà prima di iniziare, così non lo scopri dopo. **Per il Locale** serve solo saper aprire un terminale e incollarci dentro un comando. Non devi saper programmare: il codice lo scrive l'agente. Devi però saper leggere un messaggio di errore e riportarlo a lui — è il 90% del mestiere. **Per il Completo** serve qualcosa in più: devi sapere cos'è un database e cos'è una variabile d'ambiente, e devi essere a tuo agio nel muoverti dentro Claude Code — perché passerai per la creazione di un progetto Postgres, per la configurazione di un bot, per il deploy e per il debug di un webhook che non risponde. Nessuno di questi passaggi è difficile. Sono quattro cose diverse da tenere insieme, e se sono tutte nuove insieme, è lì che ci si arrende. Se leggendo hai pensato "vabbè, tanto ci pensa l'agente": parti dal Locale.

Il Percorso Completo mette i tuoi dati su internet. Patrimonio, contatti, salute, riflessioni: tutto quello che il sistema sa di te finisce su un indirizzo pubblico. Per questo il cancello con password **non è opzionale e non è una rifinitura**: si costruisce nella Parte 3, appena esistono i dati, e non si fa nessun deploy senza. La Parte 3 spiega anche perché "tanto l'indirizzo non lo conosce nessuno" non è una difesa.

Le chiavi API si pagano a consumo. La chiave Anthropic — e sul Completo anche quella OpenAI — non è inclusa in nessun abbonamento: paghi quello che usi. 🖐️ Metti un limite di spesa mensile sul pannello del fornitore **il giorno stesso in cui crei la chiave**, prima ancora di scrivere codice. Non è pessimismo: è che un ciclo sbagliato che chiama il modello a ogni render lo scopri dalla bolletta.

I dati che ci metti dentro sono tuoi, e restano dove decidi tu. Sul Locale vivono in un file sul tuo disco, e cancellarlo li fa sparire davvero. Sul Completo vivono nei servizi che hai scelto tu, e l'export della Parte 7.6 è la via d'uscita: in qualsiasi momento puoi portarli via e andartene. Un sistema personale da cui non puoi portare via i dati non è tuo del tutto.

Un'ultima onestà: il Percorso Locale l'abbiamo verificato per intero, passo per passo. Del Completo abbiamo verificato il codice; le procedure sui pannelli dei servizi esterni — BotFather, Supabase, Vercel — le abbiamo descritte con cura, ma sono pannelli che cambiano, e lì gli occhi devono essere i tuoi.

Come si leggono i marcatori

Da qui in avanti il documento usa poche marcature, sempre le stesse:

-  segnala un **bivio**: il punto in cui i due percorsi si dividono. Leggi solo il ramo del tuo.
-  solo Percorso Locale ·  solo Percorso Completo ·  entrambi. Se un blocco non è del tuo percorso, saltalo senza leggerlo.
-  riguarda solo chi è partito dal Locale e sale al Completo. Se sei nato sul Completo, non ti serve.
-  segnala un punto in cui serve un'azione **tua**, fuori da Claude Code: creare un account, copiare una chiave, cliccare in un pannello. Lì l'agente si ferma e aspetta te — se non lo fai, il pezzo dopo non può funzionare.
-  segnala un'avvertenza secca — soldi, dati o sicurezza. Vale per tutti i percorsi e non si salta.

Cosa ti serve prima di iniziare

Entrambi i percorsi

- Node.js installato
- Claude Code installato e funzionante
-  Una chiave API Anthropic, con credito e con un limite di spesa mensile

Solo il Completo, in più

-  Una chiave API OpenAI — serve per trascrivere le note vocali e per la memoria
 -  Un progetto Supabase (il piano gratuito basta)
 -  Un account Vercel (il piano gratuito basta)
 -  Un account Telegram
-

Parte 1 • Le scelte di stack

Prima di scrivere una riga di codice. Qui non c'è una lista della spesa: c'è il motivo di ogni scelta e cosa potresti usare al posto delle nostre scelte. Se un'alternativa ti convince di più, prendila — l'architettura del sistema non cambia.

Una premessa che vale per tutta la sezione: **stai scegliendo per una persona sola, tu**. Quasi tutte le decisioni che sembrano difficili diventano facili quando ricordi che non devi reggere mille utenti, non devi garantire uptime a nessuno, e se qualcosa si rompe alle 23 puoi anche andare a dormire.

Il quadro d'insieme

Livello	Percorso Locale	Percorso Completo
Framework	Next.js, App Router	uguale
Linguaggio	JavaScript	uguale
Dati	un file JSON	Supabase (Postgres + pgvector)
Modello	Claude	Claude + OpenAI (trascrizione, embedding)
Agente di coding	Claude Code	uguale
Dove gira	la tua macchina	Vercel
Cattura	barra di cattura sulla dashboard + voce del browser	+ bot Telegram
Accesso	nessuno: sei tu davanti allo schermo	password + cookie firmato

Le prime tre righe che restano identiche non sono un dettaglio: sono il motivo per cui puoi partire dal Locale e salire al Completo senza riscrivere l'applicazione. Cambia dove stanno i dati e da dove arrivano le catture. Il resto regge.

1.1 • Framework

Scegliamo: Next.js, App Router

Perché. Ti serve una cosa sola che faccia due mestieri: servire una dashboard interattiva nel browser e, allo stesso tempo, eseguire chiamate al modello lato server. La seconda parte è quella che conta davvero, e il motivo è la chiave API: **non deve mai finire nel browser**. Chiunque apra gli strumenti di sviluppo su una pagina web vede tutto quello che la pagina ha caricato. Con Next scrivi la logica del modello dentro `app/api/`, quel codice gira sul server e basta, e non hai un secondo progetto da mantenere e da mettere online.

C'è un secondo motivo, meno ovvio e più pratico: **il codice lo scrive un agente**. Quando il 90% delle righe le scrive Claude Code, quanto materiale su quel framework c'è nel mondo diventa un criterio tecnico legittimo. Next è il framework React su cui esiste più codice pubblico. Si vede: l'agente sbaglia meno, e quando sbaglia sa correggersi.

Sulla versione: non fissarti sul numero. Noi abbiamo costruito su quella che `create-next-app` installava al momento della build: la serie 16. Nei prompt trovi quel numero; se nel frattempo `create-next-app` ti installa una serie successiva, usa quella e vai avanti. Quello che usiamo qui — la cartella `app/`, i route handler in `app/api/`, la divisione tra componenti server e client — è stabile da diverse versioni. Un avvertimento concreto: le versioni recenti di Next hanno cambiato alcune API rispetto a quelle su cui i modelli sono stati addestrati, e la libreria stessa lascia nel progetto una nota che lo dice. Se l'agente scrive codice che non compila con errori strani su API che "dovrebbero esserci", digli di leggere la documentazione dentro `node_modules/next/dist/docs/` prima di riprovare. Risolve quasi sempre.

Alternative

- **SvelteKit** — stesso modello (pagine + funzioni server nello stesso progetto), bundle più leggero, sintassi più asciutta. Ecosistema più piccolo, e l'agente ne ha visto meno.
- **Remix / React Router** — solido, buon modello di caricamento dati. Nessun problema se lo conosci già.
- **Astro** — ottimo per siti prevalentemente statici. La nostra dashboard è interattiva quasi ovunque: lavoreresti controcorrente.
- **Express + React separati** — funziona, ovviamente. Ma ti ritrovi a scrivere a mano la colla che i framework sopra ti danno inclusa: build, routing, gestione delle variabili, deploy di due cose invece che di una.

1.2 · Linguaggio: JavaScript, non TypeScript

Scegliamo: JavaScript

Perché. È una scelta controcorrente e va motivata. TypeScript prende una classe di errori e te li mostra prima di eseguire il codice: è un vantaggio vero. Ma cambia natura quando il codice lo scrive un agente. Il ciclo diventa: l'agente scrive, il compilatore si lamenta di un tipo, l'agente aggiusta il tipo, il compilatore si lamenta di un altro, e tu guardi. Su un progetto personale di poche migliaia di righe quel tempo non te lo ripaga nessuno.

Il costo, dichiarato. Gli errori che non vedi in compilazione li vedi in esecuzione. Un campo che è `null` quando non te lo aspetti diventa una pagina bianca invece che una riga rossa nel terminale. Lo accetti perché sei l'unico utente e perché la pagina bianca la vedi tu, subito, non un cliente tra tre settimane.

Alternative

- **TypeScript in modalità strict** — prendila se il progetto è destinato a crescere, se ci metteranno le mani altre persone, o se semplicemente ti trovi più a tuo agio così. L'architettura non cambia di una virgola: cambiano le estensioni dei file e il tempo del primo giorno.
- **TypeScript con controlli morbidi** — via di mezzo onesta: tipi dove aiutano, `any` dove non servono.

1.3 • I dati

Qui i percorsi si dividono

Locale — un file JSON

Perché. Zero installazioni, zero migrazioni, zero servizi da tenere accesi. Il file lo apri con un editor di testo e lo leggi: quando qualcosa non torna, guardi i dati con i tuoi occhi invece di scrivere una query. E la reversibilità è totale — cancelli il file e il sistema torna com'era il primo giorno; cancelli la cartella e non è mai esistito.

Nella nostra build ci sono due file: uno con i dati iniziali che non si tocca mai, e una copia di lavoro che si rigenera da sola quando manca. È un trucco banale che vale molto: hai un "reset" affidabile senza scrivere una riga di logica.

Quando smette di bastare, per davvero. Non quando i dati crescono — un anno di catture personali in JSON non è un problema per nessun computer moderno. Smette di bastare in due casi precisi:

1. Vuoi scrivere dalla dashboard e dal telefono nello stesso momento. Due scritture simultanee sullo stesso file si sovrascrivono a vicenda.

2. **Vuoi mettere il sistema online su un hosting serverless.** E questo è il muro vero: su Vercel il filesystem è effimero, ogni funzione parte da una copia pulita del progetto e quello che scrivi sparisce. Non è una limitazione da aggirare — è come funziona. Se vuoi andare online, ti serve un database.

c Completo — Supabase (Postgres + pgvector)

Perché. È Postgres gestito da qualcun altro, con la ricerca vettoriale già dentro. Quel "già dentro" è tutto il ragionamento: la memoria semantica ha bisogno di confrontare vettori, e averla nello stesso database dei task significa un servizio solo da configurare, una connessione sola, e la possibilità di unire in una query il risultato semantico con la riga originale. L'alternativa — database normale + servizio vettoriale separato — significa due account, due chiavi, due punti di rottura e la fatica di tenere allineate due copie della stessa cosa.

Il piano gratuito copre l'uso personale. Verifica sul loro sito cosa include oggi, in particolare cosa succede ai progetti inattivi: le condizioni cambiano ed è meglio saperlo prima.

La regola che rende indolore il passaggio

Vale per entrambi i percorsi ed è l'unico consiglio architetturale di questa guida:

Tutte le letture e le scritture passano da un modulo solo.

Nella nostra build è `lib/store.js`. Nessun componente e nessuna rotta tocca i dati direttamente: chiedono al modulo. Il giorno in cui passi dal JSON a Postgres riscrivi quel file e basta. Se invece sparpagli le letture in venti componenti, quel giorno diventa un fine settimana.

Alternative

- **Neon** — Postgres serverless, supporta pgvector. Sensazione d'uso molto simile a Supabase, un po' meno "tutto incluso" (niente storage, niente auth pronta).
- **Turso / SQLite** — leggero, veloce, economico. Niente ricerca vettoriale nativa: dovresti affiancare un altro servizio, e torniamo al problema dei due pezzi.
- **Firebase** — autenticazione più facile, ricerca vettoriale più scomoda. Sensato se dell'intero sistema ti interessa tutto tranne la memoria semantica.
- **Postgres tuo su un VPS** — controllo totale, costo prevedibile, e sei tu il DBA: aggiornamenti, backup, spazio disco. Scelta legittima se un server già ce l'hai e ci sai stare dentro.

1.4 · Il modello

Scegliamo: Claude per classificare e per rispondere

Perché. Al sistema il modello serve per due lavori diversi.

Il primo è **smistare**: prende una frase buttata lì — "ho pagato 340 euro di commercialista", "ricordati di richiamare Marco" — e decide dove va archiviata, restituendo un JSON che il codice deve poter leggere senza acrobazie. Qui conta la disciplina sul formato: un modello che ogni tanto ti mette una frase di cortesia prima delle graffe è un modello che ogni tanto ti rompe la cattura.

Il secondo è **rispondere alle tue domande** sulla memoria, in italiano, citando da dove viene quello che dice. Qui conta la resa in italiano e la capacità di stare dentro i dati che gli hai dato invece di inventare.

C'è anche un motivo pratico: un fornitore solo da configurare sul Locale, e lo stesso con cui stai già costruendo. Ma qui va detta una cosa che quasi tutti danno per scontata al contrario:

l'abbonamento a Claude non ti dà la chiave API. Pro e Max coprono Claude Code — l'agente con cui costruisci — non le chiamate che il tuo sistema farà per conto suo. La chiave è separata, ha un suo credito e si paga a consumo, e ti serve **anche sul Percorso Locale**: è la stessa che trovi in cima all'Appendice B. Un account, una chiave, si comincia.

Tre tarature che ci sono costate tempo

Non sono dettagli: sono la differenza tra un sistema che si usa e uno che infastidisce.

1. **Sullo smistamento, niente ragionamento esteso.** Sull'API è una funzione opt-in: il punto non è disattivarla, è non accenderla su queste chiamate — e se il tuo setup la accende di default, qui va spenta. Classificare una frase in sette destinazioni non richiede che il modello ci pensi su, e col ragionamento attivo l'attesa può passare da secondi a molti di più. Su una barra di cattura in cui scrivi e aspetti, quella differenza è tutto.
2. **Sulle risposte, lascia margine nel tetto di token.** Il modello produce un blocco di ragionamento prima del testo, e quel blocco consuma il budget. Con un tetto basso il ragionamento se lo mangia tutto e il testo non arriva mai: l'endpoint risponde vuoto e sembra rotto. Alza il tetto e **concatena tutti i blocchi di testo della risposta, non solo il primo.**
3. **Il nome del modello sta in una variabile d'ambiente, non scritto a mano nel codice.** Nel codice resta solo un valore predefinito, per quando la variabile manca; il modello si cambia da `ANTHROPIC_MODEL`, non cercando il nome in cinque file diversi. I nomi dei modelli cambiano, e trovarli sparsi nel codice il giorno che ne cambi uno è un pomeriggio buttato.

La rete di sicurezza a regole

Sotto la chiamata al modello, la nostra build tiene un classificatore stupido fatto di espressioni regolari: se trova "euro" o "fattura" manda alle finanze, se trova "chiamare" o "rispondere" manda

a persone. Entra in funzione quando la chiave manca o l'API non risponde.

Non è ridondanza pigra, è un principio di progetto:

La cattura non fallisce mai. Al massimo smista peggio.

Se il modello è irraggiungibile e la tua nota si perde, il sistema ha appena tradito l'unica promessa che conta — che ti puoi fidare a scaricarci dentro le cose. Meglio una nota finita nella casella sbagliata che una nota mai esistita.

c Il Percorso Completo aggiunge OpenAI, e non è una preferenza

Servono due cose che Claude non fa:

- **Trascrivere l'audio.** Le note vocali da Telegram arrivano come file audio. Ti serve un modello di trascrizione — Whisper.
- **Generare gli embedding.** La ricerca semantica ha bisogno di trasformare ogni testo in un vettore di numeri, con lo stesso modello sia in scrittura che in ricerca.

È un vuoto di funzionalità, non un giudizio. Due chiavi invece di una, e ognuna fa quello che sa fare.

La regola che ti evita la bolletta

Il caricamento di una pagina non chiama mai il modello.

Le pagine leggono l'ultimo dato salvato. Chiamano il modello soltanto tre cose: una cattura, una domanda, o un pulsante di aggiornamento che premi tu. Più il cron, sul Percorso Completo. Senza questa regola, una dashboard lasciata aperta in una scheda del browser che si ricarica ogni tanto consuma budget mentre tu sei a pranzo.

Alternative

- **Solo OpenAI** — un account e una chiave per tutto, trascrizione ed embedding inclusi. Sulla qualità dell'italiano e sull'output strutturato fatti un'idea tua: prova le tue venti frasi tipiche su entrambi prima di decidere.
- **Google Gemini** — piano gratuito generoso, che su un progetto personale conta. Verifica come si comporta sull'output strutturato prima di costruirci sopra lo smistamento.
- **Modello locale (Ollama, LM Studio)** — nessun testo esce dalla tua macchina, ed è l'unico modo di avere privacy assoluta. In cambio: ti serve una macchina capace, le risposte sono più

lente, e sulle frasi ambigue la qualità cala. Per uno smistamento a sette destinazioni potrebbe bastare — provalo su cinquanta frasi vere prima di impegnarti.

- **Due fornitori, uno di riserva** — primario Claude, e se non risponde OpenAI. Sensato quando il sistema è vivo da mesi; all'inizio è complessità che non ti serve, perché la rete a regole già ti copre.

1.5 · L'agente di coding

Scegliamo: Claude Code

Perché. Legge tutto il progetto senza che glielo incolli, modifica i file al loro posto, esegue comandi e ne legge l'output. E in particolare: legge questo documento e lo segue. Le istruzioni che trovi nelle sezioni successive sono scritte per un agente che può aprire i file e lanciare i comandi da solo.

Alternative

- **Codex, Cursor in modalità agente, Aider, gli agenti dentro gli IDE** — funzionano tutti allo stesso modo per quello che serve qui.
- **Un assistente solo-chat** — si può, ma cambia il tuo lavoro: copi e incolli molto di più e i comandi li esegui tu. Metti in conto un buon 50% di tempo in più.

Due abitudini che valgono più della scelta

- **Git dal primo minuto, e un commit a ogni pezzo che funziona.** Con un agente che modifica dieci file in un colpo, "rimetti com'era prima" deve essere un comando, non un'indagine.
- **Un file di istruzioni nel progetto** (`CLAUDE.md` , o l'equivalente del tuo agente) con le tre o quattro regole di casa: che linguaggio, dove stanno i dati, cosa non toccare mai. Lo rileggerà a ogni sessione e smetterai di ripetere le stesse cose.

1.6 · Dove gira

Qui i percorsi si dividono

Locale — la tua macchina

Perché. Nessun account, nessun deploy, nessun indirizzo pubblico. Lanci il progetto, apri il browser, funziona. I dati sono su un disco che è tuo e nessuno ci può arrivare da fuori. Per capire

se questo sistema ti serve davvero, è il modo più rapido e più economico di scoprirlo.

Cosa perdi, detto chiaro. Il sistema è acceso quando è acceso il computer. Non ti arriva niente da fuori. Non si sveglia da solo la mattina. Se il valore che cerchi è "mando una nota vocale mentre sono in fila alla cassa", il Locale non te lo dà: ti fa vedere com'è fatto, non te lo fa vivere.

c Completo — Vercel

Perché. Il progetto Next si carica com'è, senza adattamenti. Le variabili d'ambiente e i segreti hanno un posto dove stare. HTTPS e dominio sono inclusi. I cron sono inclusi. Ogni commit produce un'anteprima separata, e questo — quando l'agente ti riscrive mezza dashboard — è una rete di protezione notevole. Al volume di una persona sola il piano gratuito basta.

Il vincolo da conoscere prima, non dopo. Le funzioni sono senza stato e il filesystem è effimero: niente file JSON, niente processi che restano in piedi tra una richiesta e l'altra, e un lavoro che dura più del tempo massimo consentito viene tagliato a metà. È esattamente il motivo per cui il Percorso Completo prevede un database vero.

Sui cron del piano gratuito: ci sono, ma con limiti su quanti e su quanto spesso, e l'orario non è garantito al minuto. Vai a leggere i limiti correnti prima di progettarci sopra qualcosa di delicato — cambiano.

Alternative

- **Netlify** — molto simile, stesso modello.
- **Railway, Render, Fly.io** — qui hai un contenitore vero e un processo che resta acceso. È la strada giusta se ti serve un lavoro lungo o un processo in ascolto. Partono da pochi dollari al mese.
- **Un VPS tuo** — controllo completo: cron di sistema nel tuo fuso orario, processo sempre acceso, file `.env` su un disco che è tuo, nessun limite di durata. In cambio diventi tu il sistemista: reverse proxy, certificato, aggiornamenti, backup, e la macchina che va sistemata quando si riempie il disco. La prima volta mettilci un pomeriggio.

La differenza tra queste strade non è "funziona / non funziona". È **chi fa la manutenzione**: qualcun altro in cambio di limiti, o tu in cambio di controllo. Su un sistema personale che nessuno paga per tenere in piedi, è una domanda seria.

1.7 • La superficie di cattura

È la parte del sistema che userai di più. Tutto il resto è visualizzazione: qui c'è l'unico gesto che ripeti dieci volte al giorno.

Qui i percorsi si dividono

Locale — la barra di cattura sulla dashboard, anche a voce

Perché. Zero configurazione e zero servizi di terzi. Una barra di cattura sempre presente: scrivi, invii, il modello smista, la scheda giusta si aggiorna. E il browser sa già ascoltare — con la Web Speech API premi il microfono, parli, il testo compare e prosegue nella stessa pipeline. Il momento "parlo e il sistema capisce" è reale, senza aver installato niente.

Due limiti da sapere. Il riconoscimento vocale del browser funziona bene nei browser della famiglia Chrome e in modo disomogeneo altrove. E il riconoscimento non avviene sul tuo computer: il browser manda l'audio ai server del suo fornitore. Se stai su questo percorso per motivi di riservatezza, la voce è l'unico pezzo che esce comunque — il testo, quello, puoi tenerlo in casa se usi un modello locale.

Completo — bot Telegram

Perché. Un webhook e hai finito: nessuna app da costruire, da firmare, da pubblicare o da far approvare a qualcuno. Funziona su qualunque telefono. Le note vocali arrivano come file che puoi trascrivere. Il bot può risponderti confermando dove ha archiviato, e quella conferma che torna in tre secondi è ciò che ti fa fidare abbastanza da usarlo davvero. Costa zero.

Alternative

- **Scorciatoia iOS** — chiedi a Siri, detta la nota, la scorciatoia la manda al tuo endpoint con un'intestazione segreta. Dettatura di sistema, ottima esperienza d'uso, solo Apple.
- **WhatsApp Business API** — funzionalmente equivalente a Telegram, con molta più burocrazia di mezzo. Ha senso solo se in WhatsApp ci vivi.
- **Bot Discord** — stesso modello. Sensato se Discord è già la tua casa.
- **Solo il form web, aperto sul telefono** — non sottovalutarlo: la dettatura della tastiera del telefono dentro un campo di testo copre buona parte del bisogno, e se aggiungi la dashboard alla schermata home hai un'icona come le altre. Quello che perdi rispetto a Telegram è la conferma che ti torna e il fatto di non dover aprire nulla.

1.8 · L'accesso

Una riga qui, il resto nella **Parte 3**, perché il cancello si costruisce appena esistono i dati — non quando arrivi al deploy.

Scegliamo: password singola + cookie firmato. Per un sistema con un utente solo, un login OAuth completo è impalcatura sproporzionata. Una password lunga, verificata una volta, che lascia un cookie firmato crittograficamente: sono poche decine di righe e regge il compito.

Alternative: un servizio di accesso davanti al sito (tipo Cloudflare Access) sposta il problema fuori dalla tua applicazione ed è più solido, in cambio di un servizio in più da configurare. Una rete privata (tipo Tailscale) rende il sistema semplicemente irraggiungibile da internet, ed è la soluzione più sicura di tutte se non ti serve arrivarci da dispositivi altrui. L'autenticazione completa di Supabase ha senso solo il giorno in cui gli utenti diventano più di uno.

Parte 2 · Disegna prima di costruire

Mezz'ora spesa qui te ne fa risparmiare cinque dopo.

Il motivo è pratico: se dai a un agente il compito di "fare una dashboard", te ne restituisce una qualsiasi. Se gli dai un'immagine precisa di com'è fatta, ti restituisce quella. E soprattutto, decidere l'aspetto **dopo** aver scritto il codice significa rifare il codice.

Come ottenere il mockup

Usa Claude Design, o qualsiasi strumento che ti produca dell'HTML. Quello che conta non è il come, ma **cosa deve contenere** perché sia portabile:

- **Un solo file HTML**, con il CSS dentro, senza dipendenze esterne e senza immagini remote. Le icone in SVG, scritte nel file.
- **I colori come variabili** in cima al foglio di stile. Non valori sparsi nelle regole: quando vorrai cambiare l'accento, vuoi toccare una riga sola.
- **Una sezione per ogni scheda**, ognuna con un identificativo parlante. È così che l'agente capisce dove finisce una scheda e comincia l'altra.
- **Un contenitore per la barra in alto e uno per la griglia**. Diventeranno due componenti.

Cosa mettere nella dashboard

Queste sono le quattro schermate di riferimento. Cambia pure — è la tua vita, non la mia.

Home — la schermata che guardi la mattina. Saluto e ora; chi sei e qual è il focus di oggi; le abitudini con la percentuale di completamento; il calendario della settimana; cosa è bloccato e da quanto; il polso finanziario; cosa hai mangiato; l'andamento fisico degli ultimi trenta giorni; gli obiettivi della settimana e del mese.

CRM — le persone e cosa devi a ognuna, divise per urgenza.

Finanze — patrimonio, quanto ti resta ai ritmi attuali, dove sono i soldi, cosa devi.

Review — la settimana che si chiude: cosa è andato, cosa è slittato, cosa resta aperto, le tre priorità della prossima.

Il prompt del mockup

Da incollare in Claude Design o nello strumento che preferisci — i requisiti sono quelli qui sopra. E se non usi nessuno strumento di design, questo prompt puoi darlo direttamente al tuo agente: un file HTML lo sa scrivere anche lui.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Genera il mockup di una dashboard personale in un solo file HTML, con il CSS dentro, senza dipendenze esterne e senza immagini remote; le icone in SVG, scritte nel file. I colori come variabili in cima al foglio di stile. Un contenitore per la barra in alto e uno per la griglia. Quattro schermate: Home (saluto e ora, chi sono e il focus di oggi, abitudini con percentuale, calendario della settimana, blocchi, polso finanziario, nutrizione, andamento fisico degli ultimi trenta giorni, obiettivi di settimana e mese), CRM con gli elementi da fare divisi per fascia di urgenza, ognuno con la persona collegata, Finanze, Review della settimana. Una sezione per ogni scheda, ognuna con un identificativo parlante. Dati di esempio scritti a mano.

Verifica: aprilo nel browser **senza rete**. Se si vede tutto lo stesso, è autosufficiente davvero.

Il prompt del porting

Questo secondo prompt si usa **dopo** aver creato il progetto della Parte 3 — prima non c'è dove portarlo:

PROMPT · DA INCOLLARE IN CLAUDE CODE

Ho un mockup HTML in [percorso] . Portalo in componenti React dentro components/ : un componente per la barra in alto, uno per la griglia contenitore, e uno per ogni schermata. Le schede restano sezioni dentro la schermata a cui appartengono. Prendi il CSS così com'è e mettilo in app/globals.css senza riscriverlo. Per ora usa dati scritti a mano dentro i componenti: i dati veri li colleghiamo dopo.

Perché dati finti adesso. Perché vuoi vedere se l'aspetto funziona prima di preoccuparti di dove arrivano i numeri. Se colleghi i dati subito, ogni ritocco visivo diventa un giro più lungo.

Parte 3 · Le fondamenta

Il progetto

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea un progetto Next.js (noi siamo sulla serie 16) con App Router, in JavaScript, senza Tailwind, senza ESLint e senza cartella `src`.

Poi i caratteri. Se il tuo mockup usa font specifici, vanno caricati davvero, altrimenti quello che vedi sul tuo computer non è quello che vedranno gli altri:

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggancia i font con `next/font/google` ed esponili come variabili CSS, poi usale nel foglio di stile.

Verifica: ispeziona un testo nel browser: il carattere calcolato dev'essere quello del mockup, non quello di sistema. La controprova arriva col porting: fatto coi font sbagliati, cambia faccia a tutto.



Dove stanno i dati

L Percorso Locale — un file

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea `lib/store.js` con quattro funzioni: leggere lo stato, scriverlo, aggiornarlo con una funzione, e riportarlo alle condizioni iniziali. Aggiungi una funzione che legge i log giornalieri di un intervallo di date. Lo stato vive in `data/personalos.json` e contiene anche il **profilo** — nome, ruolo, città, focus del giorno, elenco delle abitudini, obiettivo calorico giornaliero: sono dati di configurazione e la loro casa è lì. Al primo avvio, se il file non c'è, va copiato da `data/seed.json`, che non viene mai modificato e che porta già dentro il profilo di partenza. Aggiungi `data/personalos.json` al `.gitignore`.

Due file invece di uno, e c'è un motivo: `seed.json` è lo stato di partenza intatto, `personalos.json` è la tua vita. Cancellare il secondo ti riporta al primo. È il tuo tasto di

annullamento, e ti servirà più spesso di quanto pensi.

Perché un file basta. Un sistema personale, per una persona sola, genera pochissimi dati. Qualche migliaio di righe l'anno. Un database serve quando più persone scrivono nello stesso momento, quando i dati sono centinaia di migliaia, o quando ti serve una ricerca che un file non può fare. Fino ad allora, un file è più veloce da leggere, più facile da correggere a mano e impossibile da rompere.

Percorso Completo — Supabase

 Crea un progetto su supabase.com. Ti servono tre valori dal pannello: l'indirizzo del progetto, la chiave pubblica e la chiave di servizio. Attiva l'estensione `vector`, che userai per la memoria nella Parte 6. Sono passaggi tuoi: nel pannello di Supabase l'agente non entra.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea una migrazione con queste tabelle: **profilo** (una riga sola, di configurazione: nome, ruolo, città, focus del giorno, elenco delle abitudini, obiettivo calorico giornaliero), **catture** (testo grezzo, provenienza, classificazione, dove è stata smistata, via di classificazione), **task** (titolo, urgenza, priorità, tag, scadenza, persona collegata), **persone** (nome, organizzazione, tipo, metadati), **log giornalieri** (una riga per data, con un campo JSON per abitudini, pasti, obiettivi e finanze), **memoria** (testo, provenienza, e l'embedding come `vector(1536)`), **registro** (chi ha fatto cosa e quando). Su ogni tabella una colonna `user_id`, valorizzata dalla variabile d'ambiente `USER_ID`. Aggiungi una funzione SQL `match_memoria` che riceve un embedding e un numero, ordina per distanza coseno e restituisce le voci più vicine, più un indice HNSW sulla colonna dell'embedding. Attiva la protezione a livello di riga su tutte, in negazione totale: solo la chiave di servizio passa.

 **La migrazione va anche applicata, e l'agente non lo fa al posto tuo.** La via semplice: apri l'SQL Editor nel pannello di Supabase, incolla il contenuto del file di migrazione, esegui. La via da riga di comando, se la preferisci: `supabase login` (ti apre il browser), poi `supabase link`, poi `supabase db push`. Consiglio la prima — una finestra, un incolla, un pulsante — e se qualcosa non torna l'errore te lo ritrovi lì, in chiaro.

Sul campo dell'embedding. 1536 è la dimensione di `text-embedding-3-small`, il modello di embedding predefinito di questa guida. Se cambi modello, la dimensione della colonna cambia con lui: `ALTER TABLE` e rigenerazione di tutti i vettori, perché vettori di lunghezza diversa non si confrontano.

Perché una funzione SQL e non una query. L'operatore di distanza vettoriale non si esprime con le query normali del client: la ricerca vive dentro `match_memoria` e si chiama via `rpc()`. E l'indice è HNSW perché funziona anche su una tabella vuota — ivfflat, per costruirsi bene, vuole già dei dati dentro, e la tua tabella oggi è vuota.

Perché un campo JSON per abitudini e pasti. Perché sono cose che cambiano forma nel tempo. Oggi tracci sei abitudini, fra un mese ne tracci quattro diverse. Una tabella rigida ti costringerebbe a una migrazione ogni volta che cambi idea su te stesso.

c Il modulo dati sul database

La migrazione crea le tabelle, ma il codice non le sa ancora leggere. Serve l'equivalente di `lib/store.js` del Percorso Locale: un modulo unico che parla con Supabase, così tutto il resto dell'applicazione — rotte e schede — non sa e non deve sapere cosa c'è sotto. È la regola della Parte 1.3, ed è qui che sul Percorso Completo si incassa: le schede della Parte 5 leggeranno tutte da qui.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea `lib/store.js` come unico punto di accesso ai dati, sopra Supabase. Usa l'indirizzo del progetto e la chiave di servizio dalle variabili d'ambiente, e assicurati che il modulo venga importato solo da codice che gira sul server, mai da componenti client. Esponi funzioni per: leggere e aggiornare il profilo; leggere e scrivere le catture, i task, le persone, le voci di memoria e il registro; leggere e aggiornare il log giornaliero di una data, creando la riga se non esiste; e leggere i log giornalieri di un intervallo di date. Nessuna rotta e nessun componente tocca il database direttamente: tutti chiedono a questo modulo. Mantieni le stesse firme del modulo su file del Percorso Locale, così le rotte restano identiche nei due percorsi.

Verifica: fai passare una scrittura di prova dal modulo e guardala comparire nel pannello di Supabase. Poi prova una lettura con la chiave pubblica dal browser: deve tornare vuota — se torna dei dati, le regole di riga non stanno proteggendo.

c Il cancello — va fatto subito

⚠ **Solo Percorso Completo, ma si costruisce adesso** — appena esistono i dati, non quando arrivi al deploy. Se stai leggendo di fila, non saltare questa sezione con l'idea di tornarci "quando vado online": è esattamente il modo in cui si finisce online senza.

Il motivo è semplice: nel momento esatto in cui il deploy della Parte 7 finirà, il tuo patrimonio netto avrà un indirizzo pubblico. La dashboard che stai costruendo mostra, in chiaro e senza chiedere niente a nessuno: quanti soldi hai, dove sono, quanto spendi. I nomi delle persone che devi richiamare e le note che hai scritto su di loro. Le tue misurazioni di salute. Le riflessioni che hai dettato pensando che le leggessi solo tu. Un indirizzo pubblico senza cancello significa che tutto questo lo vede **chiunque conosca l'indirizzo**.

E l'indirizzo gira più di quanto pensi. Non serve che qualcuno lo indovini: basta che tu lo incolli in una chat, e l'app di messaggistica lo va a visitare da sola per generare l'anteprima. Basta che compaia in uno screenshot, in una registrazione dello schermo, in un video. Basta un servizio che raccoglie i certificati emessi — sono pubblici per come sono fatti. Le probabilità non sono zero, e la posta in gioco non è "qualcuno vede la mia lista della spesa".

Non serve un sistema di autenticazione vero — sei una persona sola. Basta una password.

Come funziona

Un middleware intercetta ogni richiesta. Se non c'è un cookie valido, si finisce sulla pagina di login. La pagina ha un solo campo, la password. Se è giusta, il server mette un cookie firmato con una chiave segreta — firmato significa che non lo puoi fabbricare a mano dal browser.

La parte pericolosa sono le eccezioni

Tre cose devono per forza passare senza cookie, perché a bussare non è un browser con una sessione — il webhook, ad esempio, resta fuori dal cancello per forza: Telegram non conosce la tua password.

Cosa	Perché deve passare	Come si protegge da sola
La pagina di login e le sue API	altrimenti non potresti mai entrare	non espone nulla
Il webhook di Telegram	bussa Telegram, non tu	intestazione con un token segreto e controllo del tuo ID utente
Gli endpoint chiamati dal cron	bussa la piattaforma	intestazione <code>Authorization: Bearer</code> con un segreto

Ogni rotta che escludi dal cancello deve avere una sua serratura. Una rotta esclusa e non protetta è peggio di nessun cancello, perché ti dà l'impressione di essere al sicuro.

Tre cose che si sbagliano tutti

1. **La password sia lunga.** Non c'è niente, di base, che impedisca a qualcuno di provarne diecimila. Una frase di quattro parole vale più di otto caratteri con un punto esclamativo.
2. **Il segreto per firmare i cookie è casuale, non scelto da te.** `openssl rand -hex 32`, e lo incolli senza guardarlo.
3. **Aggiungi `noindex`.** Con il cancello attivo i crawler non entrano comunque, ma è una riga e ti copre nella finestra in cui qualcosa è configurato male.

Il prompt

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi un cancello a password singola all'app.

- middleware che protegge tutte le rotte tranne: `/login`, `/api/auth/*`, `/api/telegram/webhook` e `/api/cron/*`
- cookie di sessione firmato con HMAC usando `AUTH_SECRET`, `httpOnly`, `secure`, `sameSite lax`, scadenza 30 giorni
- pagina `/login` con un solo campo password, che invia a `/api/auth/login`
- `/api/auth/login` confronta con `DASHBOARD_PASSWORD` usando un confronto a tempo costante e imposta il cookie
- `/api/auth/logout` cancella il cookie
- le rotte API accettano in alternativa l'intestazione `x-api-secret` confrontata con `API_SECRET`, anche questo con un confronto a tempo costante, per l'accesso da script
- intestazione `X-Robots-Tag: noindex` su tutte le risposte

Poi mostrami l'elenco esatto delle rotte escluse dal middleware e come ognuna si protegge da sola.

Quando l'agente ha finito, **fai tu l'ultima verifica**: apri l'elenco delle esclusioni e per ognuna chiediti chi può chiamarla e con cosa. È un minuto di lettura e non delegarlo.

Al momento del deploy tornerai sul cancello per due controlli specifici — sono nella Parte 7.1.

Parte 4 · La cattura

Questa parte va costruita prima delle schede. Tutto il resto legge da quello che scrive lei.

È anche la ragione per cui un sistema del genere funziona dove le app di produttività falliscono. Il problema non è mai stato dove scrivere le cose: è che nel momento in cui ti vengono in mente, aprire l'app giusta e incasellarle costa più della cosa stessa. Così non lo fai, e si perdono.

Qui il gesto è uno: dici la cosa. Il resto lo fa il sistema.

Come funziona

Arriva un testo. Va al modello con una domanda sola: *dove va a finire questa frase?* Torna indietro una destinazione, una riformulazione breve e un'urgenza. Il sistema scrive in tre posti: nell'elenco delle catture grezze, nella memoria, e nella scheda di destinazione.

Le destinazioni di riferimento sono sette, ed è l'elenco che vale in tutto il documento: **task** (il generico "da fare", per tutto quello che non ha una casa più precisa), **persone**, **finanze**, **nutrizione**, **salute**, **obiettivi**, **memoria**. Il codice valida contro questo elenco, non contro il buon senso del modello. Cambiale pure sulle tue — ma cambiale in un posto solo.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea `lib/classify.js`. Riceve un testo e restituisce `{ destinazione, titolo, persona, urgenza }`. Le destinazioni possibili sono sette: task, persone, finanze, nutrizione, salute, obiettivi, memoria. L'urgenza è una tra oggi, settimana, più avanti, con predefinito "oggi" quando il modello non si sbilancia; "in ritardo" non è mai un valore in ingresso — in ritardo ci si finisce, non ci si nasce. Usa Claude con un prompt di sistema che elenca le destinazioni possibili e chiede di rispondere solo in JSON. Il nome del modello leggilo dalla variabile d'ambiente `ANTHROPIC_MODEL`, lasciando nel codice un valore predefinito per quando manca. Non attivare il ragionamento esteso su queste chiamate, e se il tuo setup lo accende di default qui va spento: non serve e allunga l'attesa. Se la chiamata fallisce o la chiave manca, ripieghi su delle regole basate su parole chiave, così la funzione non si pianta mai. Ogni cattura salva anche **come** è stata classificata — modello primario, riserva, o regole — e la dashboard lo mostra in modo discreto sulle catture recenti.

Perché la rete di sicurezza. Perché il giorno in cui l'API non risponde, tu vuoi comunque poter buttare dentro un pensiero. Un sistema che si blocca quando internet fa i capricci è un sistema che smetti di usare.

Perché la via di classificazione va salvata. Perché una rete di sicurezza silenziosa è una bugia a fin di bene: se la chiave scade e nessuno te lo dice, il sistema continua a smistare — peggio — e tu pensi che sia il modello a essere peggiorato. Con la via scritta accanto a ogni cattura, la diagnosi è un'occhiata.

Perché niente ragionamento esteso qui. Sull'API è opt-in: il punto non è disattivarlo, è non accenderlo — e se il tuo setup lo accende di default, spegnerlo su questa chiamata. Smistare una frase non è un compito che migliora pensandoci su: lasciandolo attivo l'attesa può passare da secondi a molti di più, e nel frattempo tu stai guardando una rotellina che gira.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea la rotta di cattura. Riceve un testo, lo fa classificare, scrive nelle catture, aggiunge una voce in memoria, e se la destinazione ha una scheda dedicata aggiorna anche quella. Restituisce la destinazione, così l'interfaccia può mostrarla.

La barra di cattura

Una barra fissa in fondo alla dashboard, sempre presente. Quattro stati: a riposo, in ascolto, in elaborazione, fatto. Quando risponde, ti dice dove ha messo la cosa.

Per la voce non serve nessun servizio esterno: il browser ha il riconoscimento vocale integrato. Funziona in Chrome, ed è gratis.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi la barra di cattura fissa in fondo, con un pulsante microfono che usa il riconoscimento vocale del browser, un campo di testo, e i quattro stati. Invio manda alla rotta di cattura. A operazione finita mostra dove è stata smistata la frase, poi torna a riposo dopo un paio di secondi.



Catturare da fuori



Percorso Locale

Non c'è. Catturi dalla dashboard, anche a voce, ma devi averla aperta.

È il limite vero di questo percorso, ed è giusto saperlo: il valore più grande di un sistema così è prendere il pensiero **nel momento in cui arriva**, che di solito è mentre cammini, non mentre sei alla

scrivania.

c Percorso Completo — Telegram

Un bot Telegram è il modo più rapido di avere una casella di cattura sul telefono senza costruire un'app.

1. 🖐 Su Telegram cerca **@BotFather** e manda `/newbot`
2. Scegli un nome e un nome utente che finisca per `_bot`
3. Ti restituisce un token: è la chiave del bot
4. Genera un segreto per il webhook: `openssl rand -hex 32`
5. 🖐 Cerca **@userinfobot** e scrivigli: ti dà il tuo identificativo numerico

L'ultimo serve perché il bot risponda **solo a te**. Senza quel controllo, chiunque trovi il tuo bot può scrivere nel tuo sistema.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea la rotta del webhook Telegram. Verifica l'intestazione col segreto, verifica che il mittente sia il tuo identificativo, e scarta tutto il resto. Se il messaggio è vocale, scarica l'audio e trascrivilo con Whisper di OpenAI. Poi passa il testo alla stessa pipeline di classificazione che usa la dashboard. Rispondi su Telegram con la conferma e dei pulsanti per correggere con un tocco l'urgenza che il classificatore ha assegnato. La stessa rotta gestisce anche gli aggiornamenti di tipo `callback_query`, quelli che arrivano dai pulsanti: lì il mittente sta in `callback_query.from.id` e non nel messaggio, si risponde sempre con `answerCallbackQuery` — altrimenti il pulsante resta a girare sul telefono — e si aggiorna l'elemento già salvato.

Perché i pulsanti di correzione. Perché il modello sbaglia l'urgenza più spesso di quanto sbagli la categoria, e correggerla con un tocco mentre cammini è la differenza tra fidarsi del sistema e dover rimettere a posto tutto la sera.

🖐 Registra il webhook chiamando l'API di Telegram con l'indirizzo della tua applicazione online, passando il segreto. Questo si fa **dopo il deploy**: serve un indirizzo pubblico. Il comando è nella Parte 7.4, e lo lanci tu.

Se non vuoi Telegram, un'alternativa che non richiede terze parti è un comando rapido su iPhone che manda la trascrizione alla tua rotta di cattura. Stesso risultato, un pezzo in meno da configurare, funziona solo su Apple.

Parte 5 • Le schede

Adesso che qualcosa scrivi, le schede possono leggere.

Questa è la parte più lunga della guida, ed è normale: le schede sono il sistema, tutto il resto era preparazione. Si costruiscono **nell'ordine in cui sono scritte qui** — dalla più facile alla più difficile. Non è un ordine estetico: ogni scheda finita ti dà qualcosa che usi la sera stessa, e le prime ti fanno le ossa per le ultime.

Le regole che valgono per tutte

Prima delle schede, le regole. Sono sette. Le prime le hai già incontrate nelle Parti 1 e 3; qui diventano operative.

1 • Ogni scheda risponde a una domanda che ti fai davvero. È quello che distingue una dashboard utile da una scenografia. Se per una scheda non sai dire la domanda, quella scheda non ti serve — non costruirla.

§	Scheda	La domanda	Da dove legge
5.1	Operator + Session	chi sono, e cosa devo fare adesso	profilo + CRM
5.2	Calendario	cosa mi aspetta	Google Calendar
5.3	Abitudini	a che punto sono oggi	log giornaliero
5.4	CRM	chi aspetta qualcosa da me, e quanto è urgente	task e persone
5.5	Nutrizione	quanto ho mangiato	log giornaliero
5.6	Salute	come sta andando il mese	gli ultimi trenta log
5.7	Obiettivi	cosa mi ero promesso	obiettivi
5.8	Polso finanziario	sto salendo o scendendo	il tuo foglio di calcolo

2 • Nessuna scheda chiama un modello quando apri la pagina. Mai, ed è la regola che vale più soldi di tutte — l'hai letta nella Parte 1.4, qui si applica. Le schede leggono l'ultimo dato salvato. Il modello parte solo per quattro cose: una cattura, una domanda, un pulsante che premi tu, un lavoro pianificato. Se aprire la dashboard costa una chiamata, ogni occhiata è una spesa — e una scheda del browser lasciata aperta è una spesa che non vedi.

3 • Tutti i dati passano dal modulo unico. È la regola della Parte 3, e qui si incassa: nessuna scheda legge o scrive direttamente — ogni scheda parla con una sua rotta API, e la rotta parla col

modulo dei dati. Per questo le istruzioni di questa parte sono quasi tutte identiche per i due percorsi: la scheda non sa se sotto c'è un file o un database, e non deve saperlo.

4 · Ogni gesto risponde subito. Clicchi una spunta e la spunta appare, *poi* parte la scrittura. Se la scrittura fallisce, si rilegge lo stato vero dal server, così lo schermo non resta a raccontare una cosa mai salvata. Il motivo è di uso, non di eleganza: una dashboard in cui ogni clic aspetta il disco sembra rotta anche quando funziona, e le cose che sembrano rotte si smette di usarle.

5 · "Che giorno è oggi?" ha una sola risposta nel codice. Una funzione sola, che usa il fuso orario dell'utente (`USER_TIMEZONE`), chiamata da chiunque abbia bisogno di una data: le abitudini, i pasti, i log, le istantanee. Perché sia una trappola, lo vedrai nella scheda Abitudini; per ora basta la regola: se in due punti del codice si calcola "oggi" in due modi, prima o poi i due "oggi" saranno giorni diversi.

6 · I numeri stanno fermi. Allineati a destra, con le cifre a larghezza fissa. Sembra estetica, non lo è: senza, le colonne ballano a ogni aggiornamento e smetti di riuscire a confrontare.

7 · Il colore ha significato, non decorazione. Verde per quello che va nella direzione giusta, rosso per il contrario, arancione per quello che scade. Se lo usi anche per abbellire, hai perso il canale. E attenzione: il segno meno non è di per sé negativo — un peso che scende può essere esattamente quello che volevi. È la metrica a decidere il colore, non il segno.

Un'ultima abitudine di lavoro, che viene dalla Parte 1: **una scheda alla volta, un commit a scheda finita.** L'agente costruisce in fretta, e la tentazione di fargli fare tre schede in un colpo è forte. Resistile: quando qualcosa si rompe, vuoi sapere in quale delle tre.

5.1 · Operator + Session

Si parte da qui perché è la coppia più facile — e perché è la prima cosa che vedi ogni mattina.

La domanda. Operator: *chi sono, qual è il focus, da quanti giorni tengo.* Session: *che ore sono, che giorno è, e quali sono le tre cose che contano oggi.*

Da dove legge. Operator dal **profilo**: nome, ruolo, città e focus del giorno. Il profilo è la casa dei dati di configurazione del sistema — sul Locale una chiave del file JSON che nasce dal seed, sul Completo la riga unica della tabella omonima — e ci tornerai per le abitudini (5.3) e per l'obiettivo calorico (5.5). La striscia invece non è configurazione: si calcola dai log giornalieri delle abitudini, contando all'indietro i giorni consecutivi con almeno un'abitudine completata. Session dall'orologio e — questa è la parte che conta — **dal CRM.**

Quando ci clicchi. Operator è l'eccezione alla regola: non succede niente, ed è giusto così. È uno specchio, non un pannello di controllo. In Session invece ogni task è cliccabile e ti porta dritto alla

sua scheda nel CRM, col pannello di modifica già aperto.

I tre task di oggi

Session non è un saluto con l'ora grande: è il posto dove il sistema ti dice cosa fare. Prende dal CRM tutto quello che è **in ritardo** o **in scadenza oggi**, lo ordina, e te ne mostra tre. L'ordine è: prima la fascia (In ritardo batte Oggi), poi la temperatura (caldo batte tiepido batte freddo), poi l'ordine manuale che hai dato tu trascinando le carte nel Kanban — che vedrai nella 5.4.

Perché tre e non dieci. Tre è una decisione, dieci è una lista. Una lista la scorri e la ignori; tre cose in cima allo schermo alle otto di mattina sono un ordine di marcia. Se i task di oggi sono meno di tre, mostra quelli che ci sono. Se non ce n'è nessuno, dillo — "niente in scadenza oggi" è un'informazione, non un vuoto da riempire.

Un dettaglio tecnico che ti risparmia un errore strano: l'orologio parte **solo sul browser**. Se lo si disegna anche sul server, i due orari non coincidono mai e React protesta con un errore di hydration che sembra molto più grave di quello che è.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci le schede Operator e Session. Operator è quasi statica: legge dal profilo nome, ruolo, città e focus del giorno, e calcola la striscia dai log giornalieri delle abitudini, contando all'indietro i giorni consecutivi con almeno un'abitudine completata. Session mostra saluto in base all'ora, orologio, data per esteso, e sotto i tre task principali di oggi letti dal CRM: filtra gli elementi nelle fasce "in ritardo" e "oggi", ordinali per fascia, poi per temperatura, poi per posizione manuale, e mostra i primi tre con titolo e persona collegata. Cliccando un task si apre la schermata CRM con il pannello di dettaglio di quell'elemento già aperto. Se i task sono meno di tre mostra quelli che ci sono; se non ce ne sono, scrivilo. L'orologio si avvia solo lato client, per evitare il conflitto tra l'ora del server e quella del browser.

Verifica: nome, ruolo e focus sono i tuoi, letti dal profilo, non quelli scritti a mano nel mockup; l'orologio gira e la console del browser è pulita — nessun errore di hydration. I tre task di Session li ricontrolli a CRM finito: quello che sta in cima alla fascia più urgente deve comparire qui.

5.2 · Calendario

La domanda. Cosa mi aspetta — oggi, e nei prossimi giorni.

Da dove legge. Da Google Calendar, ma senza OAuth. Google dà a ogni calendario un **indirizzo segreto in formato iCal**: lo trovi nelle impostazioni del calendario, sotto "Indirizzo segreto in

formato iCal". È un URL che restituisce tutto il tuo calendario in sola lettura, a chiunque lo conosca.

👉 L'indirizzo lo vai a prendere tu, dalle impostazioni del calendario, e lo incolli nella variabile d'ambiente: dentro il tuo account Google l'agente non entra.

Quel "chiunque lo conosca" è il punto: **trattalo come una password**. Va in una variabile d'ambiente, mai nel codice, mai in un commit, mai in uno screenshot. Se ti scappa, dalla stessa pagina delle impostazioni lo rigeneri e quello vecchio muore.

Perché niente OAuth. OAuth significa registrare un'applicazione, superare una schermata di consenso, gestire token che scadono e si rinnovano. Per *scrivere* nel calendario servirebbe tutto questo. Per *leggerlo* — che è l'unica cosa che la scheda fa — l'indirizzo segreto dà lo stesso risultato con un decimo del lavoro. È lo scambio giusto per un sistema personale.

Quando ci clicchi. La scheda mostra una striscia dei giorni della settimana. Clicchi un giorno e vedi i suoi appuntamenti; il giorno di oggi ha un **marcatore dell'ora corrente** — una linea che ti dice a che punto della giornata sei — e la vista si porta da sola lì.

Le due cose che l'agente sbaglierà se non gliele dici

Gli eventi ricorrenti vanno espansi. Nel formato iCal, "riunione ogni lunedì alle 9" è *un solo evento* con attaccata una regola di ripetizione. Se il codice legge gli eventi senza espandere le regole, tutti i tuoi impegni ricorrenti semplicemente non compaiono — e la scheda sembra funzionare, perché gli eventi singoli ci sono. È il tipo di bug peggiore: un'assenza. L'espansione si fa su una finestra limitata (i prossimi quattordici giorni bastano), perché una regola "ogni lunedì" senza data di fine genera occorrenze all'infinito.

Il parser sia in puro JavaScript. Il formato iCal è pieno di angoli — righe spezzate, fusi orari, regole di ripetizione — e non va mai interpretato a mano. Ma nemmeno con una libreria qualsiasi: alcune delle più diffuse si appoggiano a funzioni native di Node che il bundler serverless rompe in fase di deploy, con errori che non c'entrano niente con il calendario. Nella Parte 8 c'è il caso concreto; qui basta la regola: parser iCal in puro JavaScript.

La cache breve

Il feed contiene l'intero calendario e Google lo rigenera con calma — anche a distanza di ore. Riscargarlo a ogni occhiata è lavoro sprecato su un dato che non è cambiato. Quindi: la rotta tiene il feed già interpretato **in memoria per cinque minuti** e nel frattempo risponde da lì.

Attenzione però a *dove* sta la cache: sul server, non nel browser. La risposta HTTP dice esplicitamente al browser di non conservare niente. Il motivo della distinzione: la cache in memoria scade da sola e la controlli tu; quella del browser no, e il giorno in cui il calendario "non si aggiorna" non sapresti dove guardare.

E un'onestà sul Completo, che va detta perché contraddice metà del paragrafo qui sopra: lì la memoria del modulo è **per-istanza**, e le istanze nascono e muoiono tra una richiesta e l'altra — è il vincolo della Parte 1.6. La cache quindi è un sollievo quando c'è, non una garanzia: ogni tanto il feed viene ricaricato lo stesso. Per questo la finestra resta corta e il lavoro di interpretazione resta leggero — la scheda deve reggere anche il caso in cui la cache non ci sia mai.

La scheda è la stessa nei due percorsi. Cambiano due cose intorno: dove vive la variabile d'ambiente — `.env.local` sul Locale, il pannello dell'hosting sul Completo — e quanto ti puoi fidare della cache.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la scheda Calendario. Una rotta API legge l'indirizzo iCal dalla variabile d'ambiente `GOOGLE_CALENDAR_ICAL_URL`, scarica il feed, lo interpreta con un parser iCal in puro JavaScript ed espande gli eventi ricorrenti su una finestra di quattordici giorni. Tieni il risultato interpretato in una cache in memoria per cinque minuti; la risposta HTTP porta l'intestazione che vieta al browser di conservarla. La scheda mostra la striscia dei sette giorni della settimana corrente: cliccando un giorno si vedono i suoi appuntamenti con orario, titolo e calendario di provenienza. Nel giorno di oggi disegna un marcatore all'ora corrente e porta la vista lì. Tutti gli orari si mostrano nel fuso dell'utente.

Verifica: apri la scheda e cerca un tuo impegno **ricorrente** — la riunione del lunedì, il corso del giovedì: se ci sono solo gli eventi singoli, le regole di ripetizione non vengono espanse. Poi sposta un evento su Google e ricarica: entro cinque minuti la scheda lo segue.

5.3 · Abitudini

La domanda. A che punto sono oggi.

Da dove legge. Dal log giornaliero di oggi — dove "oggi" è calcolato con la funzione unica della regola 5. Tra un attimo vedrai perché lo ripeto.

Quando ci clicchi. Ogni abitudine è un pulsante. Le abitudini sono di due tipi: la **spunta** (fatto / non fatto — l'allenamento, la lettura) e il **contatore** (quattro bicchieri d'acqua su otto, due blocchi di deep work su tre). Il clic spunta o incrementa; un contatore arrivato al massimo, cliccato ancora, riparte da zero — che è anche il modo più semplice di correggere un clic di troppo. In alto, un anello con la percentuale di completamento della giornata. Tutto risponde subito, secondo la regola 4.

La trappola del fuso orario

Le abitudini si azzerano a mezzanotte. La domanda che decide tutto è: **mezzanotte di chi?**

Il server ha un suo orologio, e sulle piattaforme serverless quell'orologio è in UTC. In Italia, la mezzanotte UTC è l'una di notte d'inverno e le due d'estate. Se "oggi" lo calcola il server, succedono due cose, entrambe silenziose: le spunte si azzerano mentre sei ancora sveglio, e quello che spunti dopo mezzanotte — la lettura prima di dormire — finisce **sul giorno sbagliato**. La striscia si spezza senza che tu abbia saltato niente, le medie della scheda Salute contano pasti nel giorno sbagliato, e nessun errore compare da nessuna parte. Te ne accorgi dopo settimane, quando ormai non ti fidi più dei tuoi stessi dati.

È una trappola e non una svista, perché il codice sbagliato *funziona* — sul tuo computer, dove server e utente sono la stessa macchina nello stesso fuso. Si rompe solo al deploy, cioè nel momento in cui hai smesso di guardare.

La difesa è la regola 5: la data di oggi si chiede a una funzione sola, che ragiona nel fuso di `USER_TIMEZONE`, e quella funzione si usa **ovunque** — la chiave del log, il momento del reset, il giorno a cui agganciare una scrittura.

La sincronizzazione

L Percorso Locale. Non c'è niente da sincronizzare: la verità è il file sulla tua macchina, e la dashboard è l'unica a leggerlo. Il reset è implicito — ogni giorno ha la sua voce nel log, e la scheda legge quella di oggi, che nasce vuota al primo clic. Il tuo computer è quasi certamente nel tuo fuso, quindi la trappola qui dorme. **Scrivi comunque la funzione della data.** Il giorno in cui salirai al Completo non ti ricorderai di questo paragrafo.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la scheda Abitudini. L'elenco delle abitudini si legge dal profilo — è lì che si configurano, non scritte a mano nel codice — e ogni abitudine è di uno di due tipi: spunta, oppure contatore con obiettivo. Lo stato del giorno vive nel log giornaliero, con la data calcolata da un'unica funzione che usa il fuso di `USER_TIMEZONE` — mai la data del server. Il clic spunta, incrementa, o azzerava un contatore pieno; la vista si aggiorna subito e la scrittura passa dalla rotta API. In alto un anello con la percentuale di completamento: le spunte valgono piene, i contatori in proporzione. A ogni nuovo giorno la scheda riparte vuota; i giorni passati restano nel log.

Verifica: spunta un'abitudine e ricarica la pagina — resta spuntata. Togli o aggiungi un'abitudine nel profilo e ricarica: la scheda cambia senza che nessuno abbia toccato il codice. Poi apri il file dei dati e controlla che la spunta sia finita nella riga del **giorno locale**, non in quella di ieri o di domani.

c Percorso Completo. Qui si aggiungono due esigenze: la stessa spunta deve vedersi **dal telefono e dal computer**, e deve comparire subito anche quando la rete è lenta. Il pattern: una copia locale nel browser per rispondere all'istante, una scrittura verso il database a ogni clic, e all'apertura una lettura degli ultimi trenta giorni che si fonde con la copia locale. Con una cautela: se la lettura d'apertura arriva *dopo* che hai già cliccato qualcosa, non deve sovrascrivere il tuo clic — dal primo gesto in poi, le risposte vecchie del server si ignorano. Sembra un dettaglio; è uno dei bug più frustranti della Parte 8.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi la sincronizzazione alla scheda Abitudini: cache in localStorage per il rendering immediato, scrittura sul database a ogni clic, e all'apertura una lettura degli ultimi trenta giorni da fondere con la cache. Dal primo gesto dell'utente in poi, marca lo stato come modificato e ignora le risposte di letture partite prima: una risposta vecchia non deve mai sovrascrivere un clic appena fatto.

Verifica: spunta un'abitudine dal telefono e guarda il computer — dopo un ricaricamento è spunta anche lì. Poi la prova cattiva: ricarica la pagina e clicca **subito**, prima che la lettura d'apertura sia tornata. La spunta deve restare; se dopo un secondo torna indietro, la risposta vecchia sta ancora sovrascrivendo il tuo clic.

5.4 · CRM

La scheda più grande. Se le altre sono una serata, questa è più d'una: costruiscila in due passaggi — prima la struttura, con il Kanban e il pannello di modifica, poi la ricerca — e verifica il primo prima di passare al secondo.

La domanda. Chi aspetta qualcosa da me, e quanto è urgente.

Da dove legge. Dai suoi elementi: impegni, quasi sempre legati a una persona. Ogni elemento ha un titolo, una nota, una **fascia di urgenza**, una **temperatura**, una persona collegata, dei tag e una posizione. È anche il posto dove la cattura della Parte 4 scarica: una nota vocale smistata come "task" o "persone" compare qui, **in testa alla fascia che corrisponde all'urgenza assegnata dal classificatore**, senza che tu abbia aperto niente.

Le fasce sono quattro: **In ritardo · Oggi · Questa settimana · Più avanti**. Perché fasce e non date di scadenza: le liste a scadenza marciscono, perché tutto slitta e ricalendarizzare è un secondo lavoro. Le fasce dicono la sola cosa che ti chiedi davvero — *quanto presto?* — e "In ritardo" non è un angolo della vergogna: è la prima cosa che Session legge la mattina.

Quando ci clicchi. Su qualsiasi elemento, si apre un **pannello laterale di modifica**: titolo, nota, fascia, temperatura, persona, tag, più i pulsanti per completare o eliminare. Esc e il clic fuori lo chiudono.

Qui c'è una trappola verificata sulla nostra build: il pannello aperto deve ricordarsi **quale elemento** sta mostrando tramite il suo identificativo — non tramite la posizione nella lista, e non tramite una copia dei dati. Il motivo: mentre il pannello è aperto, una cattura può inserire una riga in testa alla lista. Se il pannello ragiona per posizione, da quel momento sta modificando l'elemento sbagliato; se tiene una copia, mostra dati vecchi. Con l'identificativo, resta agganciato a quello giusto qualunque cosa succeda intorno.

Le tre viste

Kanban. Quattro colonne, una per fascia. Trascini una carta da una colonna all'altra per cambiarle fascia; la trascini **dentro la colonna** per riordinarla. E il riordino va salvato, perché quell'ordine non è cosmesi: è la tua priorità, ed è esattamente quello che Session usa per scegliere i tre task del mattino. Come salvarlo senza complicarsi: ogni elemento ha un numero di posizione, e quando molli una carta si riscrivono le posizioni della fascia toccata. Esistono schemi più furbi, pensati per non riscrivere migliaia di righe — tu ne hai qualche decina, e la versione semplice è quella che non si rompe.

Per persona. Gli stessi elementi, raggruppati per la persona a cui sono legati. È la vista da aprire prima di una telefonata: tutto quello che devi a quella persona, in un colpo d'occhio, invece di quattro carte sparse in quattro fasce. Gli elementi senza persona finiscono in un gruppo a parte, in fondo.

Ricerca in linguaggio naturale. Una casella di testo dove scrivi come parli: *"cosa posso chiudere in dieci minuti mentre aspetto il treno?"*, *"cosa è fermo per colpa di altri?"*. Alla pressione di Invio — **solo a Invio, mai mentre digiti**, per la regola 2 — la domanda parte verso una rotta. Quella rotta passa al modello la lista compatta degli elementi (identificativo, titolo, fascia, persona, tag) insieme alla domanda, e si fa restituire gli identificativi pertinenti. La scheda mostra solo quelli.

E la rete di sicurezza, come per la cattura: se il modello non risponde o la chiave manca, la casella si riduce a ricerca testuale semplice sugli stessi campi. Il principio è lo stesso della Parte 4: la ricerca non si rompe mai — al massimo diventa meno intelligente.

La vista scelta sopravvive al ricaricamento

La vista attiva si salva nel browser e si ripristina all'apertura. Due righe di codice. Il motivo di menzionarle: se ogni mattina la dashboard si è dimenticata come la vuoi, ogni mattina paghi un piccolo pedaggio — e i piccoli pedaggi quotidiani sono esattamente il modo in cui si smette di usare uno strumento.

Un'ultima cosa: completare un elemento lo toglie dalla lavagna ma **non lo cancella**. Resta nei dati con la sua data di completamento, e il registro ne tiene l'evento — è il materiale con cui la review di fine settimana ti dirà cosa hai chiuso davvero.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la schermata CRM. Ogni elemento ha identificativo, titolo, nota, fascia (in ritardo, oggi, questa settimana, più avanti), temperatura (caldo, tiepido, freddo), persona collegata, tag, posizione, data di creazione e di completamento. Tre viste commutabili da un selettore, con la scelta salvata nel browser e ripristinata all'apertura. Kanban: quattro colonne per fascia, trascinamento tra colonne per cambiare fascia e dentro la colonna per riordinare; al rilascio si riscrivono le posizioni della fascia toccata. Per persona: gli stessi elementi raggruppati per persona, senza-persona in fondo. Cliccando un elemento si apre un pannello laterale per modificare tutti i campi, completare o eliminare; il pannello traccia l'elemento tramite l'identificativo, mai tramite la posizione in lista. I nuovi elementi — anche quelli in arrivo dalla cattura — entrano in testa alla fascia che corrisponde all'urgenza assegnata dal classificatore. Completare non cancella: l'elemento resta nei dati con la data di completamento, e l'evento finisce nel registro.

E poi, come passaggio separato:

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi la ricerca in linguaggio naturale al CRM. Una rotta riceve la domanda, passa al modello la lista compatta degli elementi aperti (identificativo, titolo, fascia, persona, tag) con l'istruzione di restituire solo un JSON con gli identificativi pertinenti, e risponde con quelli. La casella di ricerca parte solo alla pressione di Invio, mai a ogni tasto. Scarta gli identificativi che non esistono nella lista. Se la chiamata fallisce o la chiave manca, la casella ripiega su un filtro testuale semplice sugli stessi campi, senza errori a schermo.

Il dettaglio degli identificativi da scartare non è pignoleria: ogni tanto il modello ne inventa uno. Se il codice se lo beve, la scheda mostra un elemento vuoto e tu passi mezz'ora a cercare un bug che sta in una risposta, non nel codice.

Verifica: trascina una carta in un'altra colonna, ricarica: è rimasta lì, e anche l'ordine dentro la colonna è quello che le hai dato. Apri il pannello di un elemento e, con il pannello aperto, manda una cattura dalla barra: il pannello deve continuare a mostrare lo stesso elemento. Cambia vista, ricarica: la vista è quella che avevi scelto. Poi togli la chiave API e fai una ricerca: risponde lo stesso, in modo più stupido, senza errori a schermo.

5.5 · Nutrizione

La domanda. Quanto ho mangiato oggi, e come sono messo con l'obiettivo.

Da dove legge. Dal log giornaliero di oggi — stessa funzione della data delle Abitudini, stesso fuso, stessa trappola già disinnescata. L'obiettivo calorico contro cui si misura il totale sta nel profilo, insieme al resto della configurazione.

Quando ci clicchi. Il gesto principale non è un clic: è scrivere. *"Petto di pollo con riso e una mela"* — e il modello ti restituisce calorie, proteine, carboidrati e grassi. Il pasto compare nella lista con l'orario, il totale del giorno si aggiorna, le barre dei macro pure. Cliccando un pasto si apre la modifica diretta, ed è lì che succede la cosa interessante.

L'accoppiamento nei due sensi

I quattro numeri di un pasto — calorie e tre macro — non sono indipendenti: le calorie *sono* i macro. La formula non cambia mai: **4 kcal per grammo di proteine, 4 per i carboidrati, 9 per i grassi**. La scheda rispetta questa relazione in entrambe le direzioni, e le due direzioni usano strumenti diversi. Vale la pena capire perché.

Modifichi un macro → le calorie si ricalcolano con la formula. All'istante, in locale, senza nessuna chiamata. Qui esiste una formula esatta, e dove esiste una formula, la formula vince sul modello: è istantanea, gratuita e non sbaglia mai. Chiedere a un modello di fare $4 \cdot p + 4 \cdot c + 9 \cdot f$ è pagare per un risultato peggiore.

Modifichi le calorie → il modello ridistribuisce i macro. Qui la formula non basta: da sola ha infinite soluzioni. Si potrebbe scalare tutto in proporzione, ma di solito se correggi le calorie è perché la *porzione* stimata era sbagliata — e "pasta al pesto da 550 kcal" ha una composizione plausibile che il modello conosce e una proporzione cieca no. Quindi: una rotta riceve il nome del pasto e le calorie nuove, e il modello restituisce macro coerenti.

Con una cautela: la chiamata parte **dopo che hai finito di digitare**, non a ogni tasto. Senza questa attesa, scrivere "550" genera tre chiamate — per 5, per 55 e per 550 — di cui due inutili e tutte e tre pagate.

Due onestà

I valori del modello sono **stime**, e come tali vanno marcati nell'interfaccia — un'etichetta "stima" che sparisce se correggi a mano. Sul cibo quotidiano il modello è sorprendentemente decente; per il resto una bilancia da cucina resta più precisa — ma non ce l'hai dietro al ristorante. Il valore di questa scheda è la tendenza settimanale, non la terza cifra.

E il **reset è giornaliero**: a mezzanotte — la tua — la scheda riparte vuota. I pasti di ieri non spariscono: restano nel log, ed è da lì che la prossima scheda li andrà a riprendere.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la scheda Nutrizione. I pasti del giorno vivono nel log giornaliero, ognuno con orario, nome, calorie, proteine, carboidrati, grassi e un contrassegno "stimato". Una rotta di stima riceve la descrizione testuale di un pasto e restituisce, tramite il modello, calorie e macro in solo JSON. Una rotta di ridistribuzione riceve nome del pasto e calorie di destinazione e restituisce macro coerenti. Nella scheda: campo per descrivere un pasto, totale del giorno contro l'obiettivo calorico letto dal profilo, barre dei macro, lista dei pasti. Cliccando un pasto si apre la modifica diretta: cambiando un macro le calorie si ricalcolano all'istante con la formula $4 \cdot p + 4 \cdot c + 9 \cdot f$, senza chiamate; cambiando le calorie parte, solo a digitazione conclusa, la rotta di ridistribuzione. I valori stimati sono marcati come tali finché non li correggi. La data del giorno arriva dalla funzione unica col fuso dell'utente; a ogni nuovo giorno la scheda riparte vuota e i giorni passati restano nel log.

Verifica: scrivi "petto di pollo con riso e una mela" — tornano quattro numeri plausibili, marcati come stima. Cambia le proteine a mano: le calorie si muovono all'istante e il pannello dei consumi resta fermo. Cambia le calorie: parte una chiamata sola, a digitazione finita, e la stima sparisce dai valori che hai corretto tu.

5.6 · Salute

La domanda. Come sta andando il mese — non la giornata.

Da dove legge. Dagli ultimi trenta log giornalieri. E basta: niente modello, niente servizi esterni, nessuna scrittura. È pura aggregazione di dati che le altre schede hanno già prodotto, ed è la scheda più economica delle otto — il premio per aver fatto bene la Nutrizione.

Quando ci clicchi. La scheda è una tabella: una riga per giorno registrato, con calorie, proteine, carboidrati, grassi e numero di pasti. Clicchi una riga e **si espande nei singoli pasti** di quel giorno, con orari e valori — il livello di dettaglio compare solo quando lo chiedi, e la tabella resta leggibile. In testa, le medie del periodo.

Le medie si calcolano solo sui giorni registrati

È la sola decisione non ovvia della scheda, e va presa bene: un giorno senza pasti registrati è un giorno in cui **non hai registrato**, non un giorno in cui non hai mangiato. Se le medie lo contassero come zero, ogni giorno saltato abbasserebbe la media e il tuo deficit sembrerebbe più virtuoso di

quanto sia. La scheda ti farebbe i complimenti per i giorni in cui l'hai ignorata — che è la definizione esatta di un dato che mente. Quindi: le medie dividono per i giorni con almeno un pasto, e la scheda dichiara su quanti giorni sta calcolando ("media su 22 giorni registrati"), così il numero porta con sé la propria attendibilità.

Se con la cattura registri anche il peso — "peso 78,2" smistato in salute — aggiungere una colonna è naturale. Ma non costruirla in anticipo: aggiungila quando i dati ci sono davvero.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la scheda Salute. Legge gli ultimi trenta giorni dal log giornaliero e mostra una tabella con una riga per ogni giorno che ha almeno un pasto registrato: data, calorie totali, proteine, carboidrati, grassi, numero di pasti. Cliccando una riga si espande l'elenco dei pasti di quel giorno con orario e valori; un secondo clic la richiude. In testa, le medie di calorie e macro calcolate esclusivamente sui giorni registrati — i giorni senza dati non entrano nel calcolo — con accanto l'indicazione di quanti sono. Nessuna chiamata al modello, nessuna scrittura: solo lettura e aggregazione.

Verifica: le medie in testa cambiano se aggiungi un pasto, e la dicitura dice su quanti giorni sta calcolando. Aggiungi a mano un giorno vuoto nei dati: la media non si muove di una virgola. Clicca una riga e si apre nei pasti di quel giorno; ricliccala e si richiude.

5.7 · Obiettivi

La domanda. Cosa mi ero promesso — questa settimana, e questo mese.

Da dove legge. Da due liste, settimana e mese. Ogni voce ha un nome, uno stato, e volendo un progresso ("3 di 5").

Quando ci clicchi. Tre gesti. **Spunti** un obiettivo per segnarlo fatto — resta visibile, spuntato: un obiettivo raggiunto che sparisce subito è una soddisfazione negata. **Aggiungi** dal campo in fondo a ogni lista: scrivi e premi Invio. **Rimuovi** a mano, da una X che compare passando sull'elemento. Non c'è nessun altro modo di far uscire un obiettivo dalla lista, ed è il punto della scheda.

La trappola: gli obiettivi che si azzerano da soli

Il riflesso naturale, dopo aver costruito Abitudini e Nutrizione, è trattare gli obiettivi allo stesso modo: dati agganciati a un periodo, che si rinnovano quando il periodo cambia. È esattamente il riflesso sbagliato, e il perché sta nella natura del dato.

Un'abitudine è una **misurazione**: "oggi ho letto" appartiene a oggi, e domani si riparte — l'azzeramento è il funzionamento corretto. Un obiettivo è una **promessa**: "firmare il contratto entro la settimana" non smette di esistere lunedì mattina. Se lo agganci alla settimana, lunedì la chiave cambia, la scheda si sveglia vuota, e i tuoi obiettivi non sono morti — sono rimasti attaccati alla settimana scorsa, dove non li guarderai mai più. Il danno è doppio perché non *sembra* un bug: sembra un nuovo inizio. Un obiettivo mancato che sparisce senza che tu l'abbia guardato in faccia è la scheda che lavora contro di te.

Quindi la regola: **gli obiettivi non si azzerano mai da soli**. Li chiudi tu, o li rimuovi tu. Come si traduce dipende dal percorso.

Dove vivono

L Percorso Locale. Semplice: le due liste vivono in una loro chiave del file JSON, senza nessun aggancio a date. Non c'è niente da azzerare perché non c'è nessun periodo nella struttura. La trappola qui non può proprio scattare.

c Percorso Completo. Qui l'infrastruttura rema contro: i dati quotidiani vivono nei log giornalieri, una riga per data, ed è naturale che l'agente voglia mettere gli obiettivi lì — nella riga di oggi, o in una riga "della settimana". Il trucco è **la data fissa**: gli obiettivi si salvano su una riga con una data convenzionale che non arriverà mai per conto suo — per esempio `2000-01-01`. Una riga sola, sempre la stessa, che nessun cambio di settimana o di mese può far scadere. Perché non una tabella dedicata: potresti, ed è la soluzione "pulita" — ma è una migrazione in più per due liste, mentre la riga sentinella riusa strade già costruite. È un trucco, e i trucchi si dichiarano: **scrivilo nel file di istruzioni del progetto**, così né tu né l'agente lo "correggete" tra sei mesi scambiandolo per un errore.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la scheda Obiettivi: due sezioni, "Questa settimana" e "Questo mese". Ogni voce ha nome, stato fatto/da fare e un progresso facoltativo. In fondo a ogni sezione un campo per aggiungere una voce con Invio; su ogni voce, al passaggio, una X per rimuoverla. Spuntare segna fatto e la voce resta visibile. Gli obiettivi non si azzerano mai da soli: nessuna logica legata al cambio di settimana o di mese. [Percorso Completo: le due liste si salvano nel log giornaliero su un'unica riga con data fissa convenzionale 2000-01-01, che non viene mai fatta scadere; documenta questa scelta nel CLAUDE.md del progetto.]

Verifica: aggiungi un obiettivo, spuntalo, ricarica: è ancora lì, spuntato. La prova vera è quella del tempo — sposta la data del sistema alla settimana prossima e ricarica: le liste devono essere esattamente quelle di prima. Se si sono svuotate, qualcosa le ha agganciate al periodo.

5.8 · Polso finanziario

Il pezzo forte, tenuto per ultimo. Non perché sia il più difficile da programmare — il CRM lo batte — ma perché è quello dove le regole di questa parte si pagano care se le ignori.

La domanda. Sto salendo o scendendo.

Da dove legge. Dal tuo foglio di calcolo. Quello vero — disordinato, con i fogli accumulati negli anni, le celle unite, i totali dove capita.

Partiamo dal problema, perché è il problema a giustificare tutto il resto: **la scheda finanze è quella che muore per prima in ogni sistema personale.** Muore perché tenerla aggiornata a mano è un lavoro, e i lavori senza scadenza si smette di farli. Dopo tre settimane la scheda mostra numeri vecchi, dopo sei non la guardi più, e con lei muore un pezzo di fiducia nell'intero sistema.

La soluzione non è più disciplina. È **non aggiornarla**: tu continui a tenere il tuo foglio come l'hai sempre tenuto, e il sistema lo legge da solo.

Cosa fa il modello

Il file viene letto per intero, foglio per foglio, e trasformato in griglie di testo. Tutto insieme finisce al modello con un'istruzione sola: estrai **patrimonio netto, valuta, data di riferimento e categorie** (nome, valore, tipo: liquidità, investito, debito) — e rispondi solo in JSON. Nessuna etichetta da mettere, nessun formato da rispettare, nessuna colonna da chiamare nel modo giusto.

Perché il modello e non un parser. Perché il tuo foglio non ha uno schema: ha una storia. C'è il foglio del 2023 che non usi più, il totale scritto due volte, la colonna che a metà anno ha cambiato significato. Un parser per quel foglio sarebbe un parser per il tuo passato, e si romperebbe alla prossima modifica. Il modello lo legge come lo leggerebbe una persona — ed è la prima volta, in tutta la guida, che questa capacità viene usata su un problema che prima non aveva proprio soluzione.

Due trappole di estrazione, entrambe pagate sulla build vera, entrambe da mettere **nell'istruzione**:

- **Il doppio conteggio.** Se il file ha un foglio di riepilogo e i fogli per conto, sommare tutto conta ogni euro due volte. L'istruzione lo deve vietare esplicitamente.
- **I fogli-storico.** Da una tabella con le date va presa solo la riga più recente, non la somma della colonna. Anche questo, detto esplicitamente.

E una difesa in più: nel JSON di risposta chiedi un campo **note**, dove il modello segnala le ambiguità che ha incontrato — "non è chiaro se il foglio X sia già incluso nel totale". Quel campo lo leggi tu, non il codice: le prime volte che il numero non ti torna, la spiegazione di solito è lì. Per le

prime tre esecuzioni, confronta a mano il risultato con il foglio. Poi ti fiderai — con motivo, invece che per pigrizia.

Ogni estrazione produce **un'istantanea con la sua data**, e la scheda legge sempre l'ultima. Le istantanee non si buttano: accumulandosi diventano lo storico, ed è confrontandole che la scheda calcola i delta — oggi, trenta giorni, un anno. Lo storico ti si costruisce da solo, come effetto collaterale dell'uso.

La regola ferrea

Vale per tutte le schede — è la regola 2 — ma qui smette di essere un principio e diventa una bolletta, quindi la ripeto nella sua forma dura:

Il caricamento della pagina non chiama mai il modello. La scheda legge l'ultima istantanea salvata, sempre. L'estrazione parte in due soli modi: il pulsante di aggiornamento, o il lavoro pianificato.

Il motivo dei toni: questa è la chiamata più costosa del sistema — dentro ci passa il tuo foglio intero. Agganciata al caricamento della pagina, ogni occhiata al patrimonio diventa un'estrazione completa; una scheda del browser lasciata aperta che si ricarica da sola consuma mentre sei a pranzo, e te ne accorgi dal pannello dei consumi, a cose fatte. Sull'interfaccia la differenza è un pulsante con l'ora dell'ultimo aggiornamento accanto — "aggiornato alle 9:41" — che è anche un'informazione più onesta di un numero che sembra in tempo reale e non lo è.

Da dove arriva il foglio

L Percorso Locale. Il foglio è un file sul tuo disco: un export `.xlsx` o `.csv` del tuo foglio, in una cartella che decidi tu. Se i tuoi conti vivono su Google Sheets, scaricarne l'export ogni tanto è un gesto che probabilmente fai già; se vivono in un file, sei già pronto. La rotta legge il file dal percorso configurato, e l'estrazione parte solo dal pulsante. Zero account, zero chiavi in più.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi il Polso finanziario. Una rotta legge il file del foglio di calcolo dal percorso nella variabile `FINANCE_FILE_PATH` (xlsx o csv), trasforma ogni foglio in una griglia di testo e manda tutto al modello con l'istruzione: estrai patrimonio netto, valuta, data di riferimento e categorie con nome, valore e tipo (liquidità, investito, debito); evita i doppi conteggi tra fogli di riepilogo e fogli di dettaglio; dalle tabelle con date usa solo la riga più recente; aggiungi un campo note per le ambiguità; rispondi solo in JSON. Valida il JSON e salva un'istantanea con la data. La scheda mostra patrimonio, delta rispetto alle istantanee precedenti e categorie, leggendo sempre l'ultima istantanea salvata; l'estrazione parte esclusivamente dal pulsante di aggiornamento, mai al caricamento della pagina. Accanto al pulsante, l'ora dell'ultima estrazione e il campo note se non è vuoto.

Verifica: premi il pulsante e confronta il patrimonio con quello che leggi tu sul foglio, a mano. Poi ricarica la pagina tre volte guardando il pannello dei consumi: deve restare fermo — se si muove, l'estrazione è agganciata al caricamento. E leggi il campo note: le prime volte è lì che sta la spiegazione dei numeri che non tornano.

c Percorso Completo. Il foglio resta su Google Sheets e il sistema lo legge da lì, con due differenze: l'accesso e l'automazione.

L'accesso passa da un **account di servizio** — un'identità tecnica di Google con una sua email, a cui dai accesso in sola lettura al foglio come faresti con una persona. La strada corta che devi rifiutare è "pubblica sul web": genera un indirizzo pubblico, e un foglio con dentro il tuo patrimonio su un indirizzo pubblico è esattamente il tipo di scorciatoia di cui la Parte 3 spiega il costo. 🙅 La procedura è tutta tua — sono pannelli Google, e l'agente lì non arriva. Una volta sola:

1. 🙅 Su `console.cloud.google.com` crea un progetto e attiva le API di Drive e Sheets
2. 🙅 Crea un account di servizio e genera la sua chiave JSON
3. 🙅 Apri il foglio → Condividi → incolla l'email dell'account di servizio (finisce in `iam.gserviceaccount.com`) come Visualizzatore
4. Dal JSON, l'email e la chiave privata finiscono nelle variabili d'ambiente — e occhio agli a-capo della chiave: la Parte 7.3 spiega la trappola

È la parte noiosa della scheda, e non c'è modo di renderla interessante: sono pannelli Google, uno dopo l'altro. Fatto una volta, non ci torni più.

L'automazione è il **cron notturno**: un lavoro pianificato che chiama la stessa rotta del pulsante, una volta al giorno. Da quel momento l'istantanea è sempre fresca senza che tu preme niente, lo

storico cresce da solo, e il briefing del mattino può dirti come stai messo. La registrazione del cron e la sua protezione col segreto sono nella Parte 7 — la rotta è la stessa, cambia solo chi la chiama.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi il Polso finanziario con lettura da Google Sheets. Autenticazione con account di servizio (variabili `GOOGLE_SHEETS_FINANCE_ID`, `GOOGLE_SERVICE_ACCOUNT_EMAIL`, `GOOGLE_SERVICE_ACCOUNT_KEY`), mai con la pubblicazione sul web. La rotta scarica il file, trasforma ogni foglio in una griglia di testo e manda tutto al modello con l'istruzione: estrai patrimonio netto, valuta, data di riferimento e categorie con nome, valore e tipo (liquidità, investito, debito); evita i doppi conteggi tra fogli di riepilogo e fogli di dettaglio; dalle tabelle con date usa solo la riga più recente; aggiungi un campo note per le ambiguità; rispondi solo in JSON. Valida il JSON e salva un'istantanea giornaliera nel log. La scheda legge sempre l'ultima istantanea e mostra patrimonio, delta e categorie; l'estrazione parte solo dal pulsante di aggiornamento o dal cron notturno, mai al caricamento della pagina. Accanto al pulsante, l'ora dell'ultima estrazione e il campo note se non è vuoto.

Verifica: premi il pulsante e confronta il patrimonio col foglio, a mano — le prime tre volte, non solo la prima. Togli la condivisione all'account di servizio e riprova: deve fallire con un errore leggibile, non con un numero sbagliato. E ricaricando la pagina il consumo resta fermo: legge l'ultima istantanea, non il foglio.

Le superfici che restano

Tre superfici della dashboard non hanno avuto una sezione, e il motivo è che non introducono niente di nuovo: sono le otto schede che hai appena costruito, ricombinate.

Blocchi (nella Home) è un filtro sul CRM: gli elementi in ritardo, con la persona che li tiene fermi e da quanti giorni. Nessun dato nuovo — una lettura e un ordinamento per anzianità. Il valore è tutto nella domanda a cui risponde: *cosa è fermo, e per colpa di chi*.

La schermata Finanze è il Polso con più spazio: le stesse istantanee, aperte in liquidità, investito e passività, con lo storico mensile in tabella. Stessa fonte, stessa regola ferrea, più righe.

La Review chiude la settimana leggendo quello che le altre schede hanno scritto: cosa hai completato (i task chiusi del CRM), cosa è slittato, chi non ha mai risposto, come sono andate abitudini e pasti. Un'aggregazione più una chiamata al modello per il riassunto — dal pulsante sul Locale, dal cron del venerdì sul Completo, con lo stesso stampo del briefing del mattino della Parte 7. La versione automatica è la prima voce della Parte 9, ed è lì che trova il suo spazio.

E adesso la parte onesta: queste tre restano coi dati del mockup. Nessun prompt le collega, e non è una dimenticanza — è l'ordine che abbiamo scelto: prima le schede vive, quelle in cui scrivi e da cui leggi ogni giorno, e le viste d'insieme dopo, quando i dati sotto ci sono davvero e sai cosa vuoi vederci. Completarle è una serata e non richiede niente di nuovo, come hai appena letto. Il posto dove si fa è la Parte 9 — la Review ne è già la prima voce.

Quando l'ultima scheda legge i tuoi dati veri, fai un giro completo: una cattura dalla barra, e guardala comparire nel CRM e in Session. È il momento in cui i pezzi smettono di essere schede e cominciano a essere un sistema — e da qui si va alla Parte 6, dove il sistema comincia a ricordare.

Parte 6 · La memoria

È la parte che trasforma un'agenda in qualcosa che ti conosce.

Ogni cosa che passa dal sistema — una cattura, un task chiuso, una riga di diario — lascia una traccia. Dopo qualche settimana quella traccia diventa un contesto che nessuna app potrà mai avere su di te, perché nessuna app ti ha ascoltato per un anno.

Come cercare dentro la memoria

Percorso Locale — passi tutto

Con qualche centinaio di voci, la cosa più semplice è anche la migliore: dai al modello **tutto** il contesto e lascia che sia lui a trovare il filo.

Non è una scorciatoia. È spesso meglio della ricerca per somiglianza, perché il modello vede il quadro intero invece di dieci frammenti scelti da un algoritmo che non sa cosa stai chiedendo.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi la rotta delle domande. Raccogli profilo, memoria, catture recenti, obiettivi, persone e salute, passali tutti al modello insieme alla domanda, e chiedi di rispondere citando sempre da quale voce arriva ogni affermazione. Se un dato non c'è, deve dirlo invece di inventarlo.

La citazione delle fonti non è un vezzo. È l'unica difesa che hai contro una risposta inventata che suona benissimo. Se non può dirti da dove viene una cosa, quella cosa non c'era.

Percorso Completo — ricerca per somiglianza

Quando le voci diventano migliaia, passarle tutte smette di funzionare: costa e supera lo spazio disponibile.

Allora ogni testo viene trasformato in una lista di numeri che ne rappresenta il significato. Quando fai una domanda, la trasformi allo stesso modo e cerchi le venti voci numericamente più vicine. Quelle sono i tuoi ricordi rilevanti.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi lo strato di memoria. A ogni scrittura, calcola l'embedding del testo e salvalo. Il modello di embedding leggilo dalla variabile `OPENAI_EMBEDDING_MODEL`, lasciando `text-embedding-3-small` come predefinito nel codice. Aggiungi una rotta di ricerca che trasforma la domanda con lo stesso modello, chiama via `rpc()` la funzione `match_memoria` creata nella migrazione — l'operatore di distanza non si esprime con le query normali del client — e restituisce le venti voci più vicine con la riga di origine.

E poi la rotta delle domande, che sopra questa ricerca ci si appoggia:

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea la rotta delle domande — o adatta quella esistente, se arrivi dal Locale. Invece di passare tutto il contesto, usa la rotta di ricerca per prendere le venti voci più vicine alla domanda e passa al modello solo quelle, insieme al profilo. Le regole restano le stesse: citare sempre da dove arriva ogni affermazione, e se un dato non c'è dirlo invece di inventarlo.

Quando aggiungerlo davvero. Non subito. Se la tua dashboard guarda soprattutto all'oggi, non ti serve: aggiungilo quando ti accorgi di fare domande su un anno di dati, non su una settimana.

Chiedere al sistema

Non serve una schermata nuova. La barra di cattura che hai già può fare entrambe le cose.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Fai in modo che la barra riconosca le domande: se il testo finisce con un punto interrogativo o comincia con una parola interrogativa, invece di archiviare manda alla rotta delle domande e mostra la risposta espandendosi verso l'alto. Chiusura con la X e con Esc.

Perché nello stesso posto. Perché è come funziona nella tua testa: non decidi in anticipo se quello che stai per dire è un'informazione o una domanda. Lo dici e basta.

Parte 7 · Metterlo online

C Solo Percorso Completo

Sul Percorso Locale questa parte non ti riguarda: il tuo sistema vive sulla tua macchina e non ha un indirizzo. Prima di saltare alla Parte 8, però, chiudi il tuo percorso con la lista qui sotto — è la gemella della 7.7, senza le voci che parlano di internet.

L La chiusura del Percorso Locale

- ☐ La dashboard gira su localhost e mostra i tuoi dati, non quelli del mockup
- ☐ Scrivo una frase nella barra → finisce nella destinazione giusta, e la scheda si aggiorna
- ☐ Chiudo tutto, riavvio il computer, riapro → i dati sono ancora lì
- ☐ Cancello `data/personalos.json` e ricarico → la dashboard riparte dal seed, pulita
- ☐ So dov'è il mio backup: è la copia di quel file, e la faccio io
- ☐ Guardo il consumo sul pannello di Claude → i numeri hanno senso, e non si muovono quando ricarico una pagina

E qui hai finito. Non è una versione ridotta di qualcosa: è il sistema, che funziona, sulla tua macchina. La Parte 8 ti serve quando si rompe, la Parte 9 quando vorrai di più — e se un giorno vorrai salire al Completo, la procedura è la voce 7 della Parte 9, travaso dei dati compreso.

C Da qui in poi si va online

Prima di iniziare, la dashboard deve già girare in locale leggendo e scrivendo su Supabase, non sul file JSON. Se sei ancora sul file, il deploy non ti darà un errore chiaro: ti darà un sistema che sembra funzionare e perde i dati a intervalli casuali. È il tipo di problema che costa un pomeriggio a capire.

7.1 · Il cancello: verifica prima del deploy

Il cancello con password l'hai costruito nella **Parte 3** — è lì, per esteso, con il prompt e la tabella delle eccezioni. Se l'hai saltato, fermati: **nel momento esatto in cui il deploy finisce, il tuo patrimonio netto ha un indirizzo pubblico**. Torna alla Parte 3 e costruiscilo adesso; il deploy può aspettare mezz'ora.

Se c'è già, due controlli che riguardano solo questo momento:

1. **Le variabili d'ambiente vanno impostate anche per gli ambienti di anteprima.** Su Vercel ogni commit può generare un'anteprima con un suo indirizzo pubblico. Se i segreti li hai messi

solo in produzione, quelle anteprime girano senza cancello, o si rompono in un modo che ti viene voglia di "sistemare al volo" disattivando il controllo.

2. **Se hai già fatto un deploy senza cancello, considera l'indirizzo bruciato.** Metti il cancello e cambia il nome del progetto o il dominio. È rapido, e ti toglie il dubbio.

7.2 • Il deploy

Due strade, stesso risultato.

Dalla riga di comando, la più rapida per la prima volta:

```
npm i -g vercel
vercel login      # 🖐️ apre il browser: questo lo fai tu
vercel link       # collega la cartella a un progetto
vercel --prod     # pubblica
```

🖐️ `vercel login` apre una pagina nel browser e aspetta che tu confermi: dentro una sessione dell'agente non si conclude. Lancialo tu nel terminale, una volta sola; da lì in poi gli altri comandi funzionano anche per lui.

Da GitHub, che è dove finirai comunque: collegi il repository al progetto e ogni push sul ramo principale pubblica. Ti dà lo storico dei deploy e il ritorno alla versione precedente con un clic — che è esattamente la cosa che vuoi avere il giorno in cui una modifica rompe tutto e sono le undici di sera.

Prima di pubblicare, controlla due cose in locale:

```
npm run build     # deve finire senza errori
git status        # nessun .env, nessuna chiave, nessun file di dati
```

La build di produzione è più severa di quella di sviluppo: capita spesso che passi `npm run dev` e si fermi `npm run build`. Meglio scoprirlo qui.

7.3 • Le variabili d'ambiente

L'elenco completo è nell'**Appendice B**. Qui il metodo e le trappole.

🖐️ **Il modo consigliato è il pannello web di Vercel.** Si vedono tutte insieme — il che aiuta a non dimenticarne una — e ogni valore lo spunti per tutti gli ambienti in un colpo solo.

Dalla riga di comando esiste l'equivalente, ma `vercel env add` è un comando **interattivo**: il valore te lo chiede da tastiera. Se lo usi, usalo tu, non l'agente. E una variabile alla volta significa una riga per ambiente:

```
vercel env add ANTHROPIC_API_KEY production
vercel env add ANTHROPIC_API_KEY preview
vercel env add ANTHROPIC_API_KEY development
```

Tre righe per una sola variabile: è il punto 3 qui sotto, ed è anche il motivo per cui il pannello ti fa risparmiare la serata.

Quattro cose da sapere:

1. **Il prefisso** `NEXT_PUBLIC_` **significa "finisce nel browser"**. Non è una convenzione di nomi: è un comportamento. Qualunque variabile con quel prefisso viene inserita nel codice che scarica il browser ed è leggibile da chiunque. La chiave di servizio del database e le chiavi delle API non hanno **mai** quel prefisso.
2. **Dopo aver cambiato una variabile devi rifare il deploy**. Le variabili vengono lette al momento della build. Cambiarle e aspettarsi che l'app se ne accorga da sola è una delle mezz'ore più stupide che si possano perdere.
3. **Imposta i valori per tutti gli ambienti**, non solo produzione — è la trappola degli ambienti di anteprima vista nella 7.1.
4. **La chiave privata di un account di servizio contiene degli a-capo**. Se ti serve (per esempio per leggere il foglio delle finanze della Parte 5.8), incollala con attenzione: gli a-capo mangiati sono la causa numero uno di "chiave non valida" con la chiave giusta.

7.4 • Registrare il webhook di Telegram

Finché non fai questo passaggio, il bot esiste ma nessuno gli dice dove consegnare i messaggi.

👉 Questo comando lo lanci tu, e prima esporta i valori: le variabili che stanno nel `.env` o nel pannello di Vercel **non esistono nella tua shell**. O le esporti, o incolli i valori a mano dentro il comando — non c'è una terza via, e il segno di quando te ne dimentichi è un `secret_token` vuoto che sembra funzionare.

```
export TELEGRAM_BOT_TOKEN=...
export TELEGRAM_WEBHOOK_SECRET=...
export NEXT_PUBLIC_APP_URL=https://il-tuo-dominio

curl -F "url=$NEXT_PUBLIC_APP_URL/api/telegram/webhook" \
  -F "secret_token=$TELEGRAM_WEBHOOK_SECRET" \
  "https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/setWebhook"
```

Il `secret_token` è la parte che conta: Telegram lo rimanderà a ogni chiamata in un'intestazione, e il tuo endpoint deve rifiutare tutto ciò che non lo porta. Senza, quell'indirizzo è una porta aperta che scrive nel tuo database.

Verifica che sia andata:

```
curl "https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/getWebhookInfo"
```

Guarda i campi degli errori recenti: se il tuo endpoint sta rispondendo male, è lì che lo vedi, e ti risparmia mezz'ora di tentativi al buio.

Tre cose che fanno perdere tempo:

- **Il webhook vuole un indirizzo pubblico.** In locale non arriva niente: o usi un tunnel, o testi direttamente sul deploy.
- **Un bot può puntare a un indirizzo solo.** Se registri il bot di sviluppo sul dominio di produzione, i messaggi vanno lì e basta. Se ti serve provare senza disturbare, fai un secondo bot: costa trenta secondi da BotFather.
- **Rispondi sempre 200, anche quando qualcosa va storto.** Se rispondi con un errore, Telegram riprova, e ti ritrovi la stessa nota archiviata cinque volte. Gestisci l'errore dentro, rispondi bene fuori.

7.5 • Il cron del briefing del mattino

È il momento in cui il sistema smette di essere un posto dove vai e diventa una cosa che ti cerca. Alle sette ti arriva su Telegram: cosa hai in calendario, le priorità, cosa è slittato da ieri.

Si dichiara in `vercel.json`, alla radice del progetto:

```
{
  "crons": [
    { "path": "/api/cron/briefing", "schedule": "0 5 * * *" }
  ]
}
```

L'orario è in UTC. Sempre. In Italia siamo un'ora avanti d'inverno e due d'estate: `0 5 * * *` sono le 6 del mattino in inverno e le 7 in estate. Non c'è modo di dirgli "le 7 ora italiana": o scegli l'orario che ti va bene tutto l'anno, o due volte l'anno cambi una cifra. Segnatelo, perché quando l'ora legale si sposta e il briefing arriva un'ora prima si perde tempo a cercare un bug che non c'è.

L'endpoint deve fare tre cose:

1. **Verificare l'intestazione** `Authorization: Bearer <CRON_SECRET>`. La piattaforma la manda da sola. Senza questo controllo, quell'indirizzo è un pulsante pubblico che consuma il tuo budget di API a ogni clic.
2. **Essere idempotente.** Se per qualsiasi motivo parte due volte, tu ricevi due briefing identici. Salva da qualche parte che oggi è già stato inviato e controlla prima di mandare.
3. **Finire in fretta.** Le funzioni serverless hanno un tempo massimo. Un briefing è una manciata di query e una chiamata al modello: se ci mette più di qualche secondo, c'è qualcosa che non va nelle query, non nel modello.

Verifica i limiti del tuo piano su quanti cron puoi avere e con che frequenza — sul piano gratuito sono contati.

PROMPT • DA INCOLLARE IN CLAUDE CODE

Costruisci l'endpoint del briefing del mattino in `/api/cron/briefing`.

1. Verifica l'intestazione Authorization: Bearer con `CRON_SECRET`; se non corrisponde rispondi 401 e non fare altro.
2. Controlla se il briefing di oggi è già stato inviato; se sì esci subito.
3. Raccogli: eventi di oggi, task con urgenza "oggi", obiettivi della settimana, cosa risulta slittato da ieri.
4. Una sola chiamata a Claude che trasformi questi dati in un messaggio breve, in italiano, massimo 10 righe, senza convenevoli.
5. Invia su Telegram con `sendMessage` a `TELEGRAM_USER_ID`.
6. Registra l'invio nel registro, così una seconda esecuzione non manda un doppione.
7. Rispondi sempre 200 con un riepilogo di cosa è stato fatto.

Aggiungi la voce in `vercel.json` con schedule `"0 5 * * *"` e scrivi nel codice un commento che ricordi che l'orario è UTC.

Provalo a mano prima di aspettare domattina — e vale l'avvertenza della 7.4: nella tua shell quelle variabili non ci sono finché non ce le metti tu.

```
export CRON_SECRET=...
export NEXT_PUBLIC_APP_URL=https://il-tuo-dominio

curl -H "Authorization: Bearer $CRON_SECRET" "$NEXT_PUBLIC_APP_URL/api/cron/briefing"
```

7.6 • Il backup

Due domande a cui devi saper rispondere: *se domani il servizio del database sparisce, cosa mi resta?* e *se una modifica cancella metà dei dati, come torno indietro?*

Non dare per scontato che ci pensi il piano gratuito. Le politiche di backup e conservazione cambiano, e i progetti inattivi possono essere sospesi. Vai a leggere cosa include il tuo piano oggi, non cosa includeva quando è stata scritta la guida che stai leggendo.

Fatti il tuo export, che è mezz'ora di lavoro:

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi un endpoint `/api/admin/export` che:

- accetta solo richieste che portano un cookie di sessione valido: qui l'intestazione `x-api-secret` **non** vale, e va rifiutata esplicitamente — è la rotta che restituisce tutto il database, e la porta di servizio lì non entra
- legge tutte le tabelle e restituisce un JSON unico con dentro tutto, più la data di generazione e la versione dello schema
- imposta le intestazioni per farlo scaricare come file

Aggiungi un pulsante "Scarica backup" nelle impostazioni della dashboard che chiama questo endpoint.

Poi scegli come conservarlo:

- **A mano, ogni tanto.** Un clic, il file finisce nel tuo cloud. Meglio di niente e onestamente sufficiente all'inizio.
- **Un cron settimanale** che scrive il dump da qualche parte — un repository privato va benissimo, e in più ti dà lo storico versionato gratis. Non può passare dalla rotta qui sopra, che vuole la tua sessione: gli dai una rotta sua sotto `/api/cron/`, protetta col `CRON_SECRET`, che riusa la stessa funzione di dump.

C'è un vantaggio secondario che vale il tempo: quell'export è anche la tua **via d'uscita**. È quello che ti permette di lasciare Supabase, o Vercel, o entrambi, senza ricominciare da zero. Un sistema personale da cui non puoi portare via i dati non è tuo del tutto.

7.7 · La lista di verifica dopo il deploy

Dieci minuti, una volta sola, e dormi meglio.

- ☐ Apro l'indirizzo in una finestra anonima → **mi chiede la password**
- ☐ Provo una password sbagliata → non entro
- ☐ Entro con quella giusta → la dashboard carica i dati veri
- ☐ Chiamo un endpoint API senza cookie e senza segreto → mi risponde 401
- ☐ Chiamo l'export con il solo segreto `x-api-secret`, senza cookie → mi risponde 401
- ☐ Mando un messaggio di testo al bot → arriva la conferma e la scheda si aggiorna
- ☐ Mando una nota vocale al bot → viene trascritta e smistata
- ☐ Chiamo il webhook a mano senza il token segreto → viene rifiutato

- ❑ Chiamo l'endpoint del cron senza `Bearer` → viene rifiutato
- ❑ Chiamo l'endpoint del cron con `Bearer` → arriva il briefing su Telegram
- ❑ Scarico un backup e lo apro → dentro ci sono i dati che mi aspetto
- ❑ Cerco su Google `site:il-mio-dominio` → non trova niente
- ❑ Guardo il consumo sui pannelli di Claude e OpenAI → i numeri hanno senso

L'ultima voce ricontrollala dopo una settimana di uso vero. È l'unico modo di sapere quanto ti costa **il tuo** uso, invece di quanto costa quello di qualcun altro. Se il numero ti sorprende, quasi sempre la causa è una: qualcosa chiama il modello al caricamento di una pagina.

7.8 · Se preferisci un VPS

Vercel non è un requisito: è la strada che chiede meno tempo. Su una macchina tua cambiano cinque cose e l'applicazione resta identica. **Dichiarato subito: quella strada questa guida non te la copre passo per passo** — la tabella qui sotto ti dice cosa cambia e cosa ti prendi in carico, il resto lo costruisci con il tuo agente, che di reverse proxy e certificati ne ha visti parecchi.

Cosa	Su Vercel	Su un VPS
Il processo	funzioni che nascono e muoiono	un processo sempre acceso, con un gestore che lo riavvia se cade
L'indirizzo e il certificato	inclusi	reverse proxy + certificato che rinnovi (automatizzabile, ma lo configuri tu)
I lavori pianificati	<code>vercel.json</code> , orari in UTC	il cron di sistema, nel tuo fuso orario
I segreti	pannello del servizio	un file <code>.env</code> sul disco, con permessi stretti
Manutenzione	non tua	tua: aggiornamenti, spazio disco, backup

Il vantaggio reale non è il costo: è che un processo sempre acceso non ha limiti di durata, il cron gira nel tuo fuso senza acrobazie mentali, e le chiavi stanno su un disco che è tuo. Il costo reale è che il pomeriggio in cui configuri tutto è un pomeriggio in cui non stai costruendo il sistema — e ogni pomeriggio successivo in cui qualcosa si aggiorna, uguale.

Il cancello con password serve identico, per gli stessi motivi. Su una macchina tua non è più sicuro: è pubblico esattamente allo stesso modo.

Parte 8 · Quando si rompe

Ogni voce di questa parte è un errore incontrato davvero, costruendo il sistema che hai in mano o uno molto simile. Per ognuna trovi tre cose: quello che vedi sullo schermo, il motivo vero, e come si sistema. Leggila una volta adesso, di fila: quando l'errore arriva, riconoscere il sintomo è metà della soluzione.

Prima di tutto: come si dà la caccia a un bug con un agente

Non sei tu che correggi il codice — è l'agente. Il tuo lavoro è dargli l'informazione giusta, e c'è un modo giusto di farlo.

Riporta il messaggio d'errore intero. Non "mi dà un errore di rete": copia tutto, dalla prima riga all'ultima — il nome del file, il numero di riga, il codice di stato, lo stack completo. Sembra rumore, ed è esattamente quello su cui l'agente lavora. Il riassunto che fai tu gli toglie i dettagli che gli servivano.

Sappi dove guardare. Gli errori del server compaiono nel terminale dove gira il progetto. Gli errori del browser compaiono nella console (tasto destro sulla pagina → Ispeziona → Console). Un errore vive in uno solo dei due posti: se nel terminale non c'è niente, guarda la console, e viceversa.

Dì cosa stavi facendo e cosa ti aspettavi. "Ho scritto una frase nella barra e ho premuto invio; mi aspettavo la conferma, invece la barra è rimasta su Elaborazione." Due righe così valgono più di dieci minuti di descrizione del codice.

Fagli scrivere un test, invece di fidarti. L'agente tende a dichiarare risolto — "fatto, ora funziona" — anche quando ha solo spostato il punto in cui si rompe. La contromossa è chiedergli: *scrivi uno script che riproduce il problema, fammelo vedere fallire, poi correggi e fammelo vedere passare.* Costa un minuto in più e cambia tutto: il "funziona" smette di essere un'opinione e diventa un comando che puoi rilanciare tu, anche tra un mese.

Un bug alla volta, e un commit prima di ogni caccia. Se la correzione peggiora le cose — succede — torni indietro con un comando invece che con un'indagine.

Il modello

La risposta alle domande arriva vuota [L+C](#)

Cosa vedi. Fai una domanda al sistema e non torna niente: la rotta risponde, ma il testo è vuoto. Sembra rotto l'endpoint, e invece l'endpoint sta facendo esattamente quello che gli hai chiesto.

Perché succede. Il modello produce un blocco di ragionamento prima del testo, e quel blocco consuma il tetto di token della risposta. Con un tetto basso, il ragionamento se lo mangia tutto e il testo non arriva mai: il modello si ferma per limite di token avendo prodotto solo il pensiero, non la risposta. C'è anche una seconda trappola nella stessa voce: la risposta del modello è una **lista di blocchi** — ragionamento prima, testo poi, a volte più blocchi di testo. Se il codice prende solo il primo blocco, prende il ragionamento, o solo un pezzo.

Come si risolve. Alza il tetto — nella nostra build siamo saliti a 3000 token — e concatena **tutti** i blocchi di testo della risposta, filtrando via quelli di ragionamento. Se vuoi vederlo coi tuoi occhi, fai loggare all'agente il motivo di stop della risposta: quando è il limite di token, la causa è questa.

La barra di cattura ci mette diversi secondi a smistare L+C

Cosa vedi. Scrivi una frase, premi invio, e resti a guardare la rotellina. Ogni volta. Il sistema funziona, ma a quel ritmo smetti di usarlo.

Perché succede. Il ragionamento esteso è acceso anche sulla classificazione. Smistare una frase in sette destinazioni non è un compito che migliora pensandoci su: il modello ci pensa comunque, e tu aspetti.

Come si risolve. Sull'API il ragionamento esteso è opt-in: su questa chiamata non va attivato, e se il tuo setup lo accende di default, qui va spento. L'attesa scende dai molti secondi ai pochi — i numeri esatti dipendono dal modello e dalla giornata, la differenza si sente comunque. Sulle risposte alle domande invece lascialo stare: lì il tetto alto della voce precedente serve proprio a fargli spazio.

Smista peggio da giorni, e non c'è nessun errore L+C

Cosa vedi. Le catture finiscono nel posto sbagliato più di prima. Nessun errore, nessuna rotellina infinita, nessuna riga rossa: il sistema fa il suo lavoro, solo peggio. È il guasto più subdolo di tutti, perché sembra un peggioramento del modello.

Perché succede. Il modello non c'entra: probabilmente non lo stai nemmeno chiamando. La rete di sicurezza a regole è entrata in funzione — chiave scaduta, credito finito, nome del modello sbagliato dopo un cambio — e sta smistando lei, in silenzio, come è stata progettata per fare.

Come si risolve. Guarda la **via di classificazione** salvata accanto alle catture recenti (Parte 4): se dice "regole" da giorni, la diagnosi è fatta e le cause da controllare sono tre — la chiave, il credito, il nome del modello nella variabile d'ambiente. È esattamente per questo che la via di classificazione si salva.

Una cattura ogni tanto fallisce, sempre su frasi diverse L+C

Cosa vedi. Nove catture su dieci vanno. Una fallisce, senza uno schema riconoscibile, e nel log c'è un errore di parsing del JSON.

Perché succede. Hai chiesto al modello di rispondere solo in JSON, e lui *quasi* sempre lo fa. Ogni tanto ci mette una frase di cortesia prima delle graffe, o un commento dopo. Il tuo `JSON.parse` legge la frase di cortesia e si rompe.

Come si risolve. Non fidarti che la risposta sia solo JSON: estrai quello che sta tra la prima graffa aperta e l'ultima chiusa, e fai il parsing di quello. Due righe di codice, e la classe di errori sparisce. E se il modello che usi supporta l'**output strutturato** — quello che si ottiene dichiarando uno schema come strumento — usalo: elimina il problema alla radice invece di ripulirlo a valle. L'estrazione tra graffe resta comunque, come rete di sicurezza: costa due righe e ti copre il giorno in cui cambi fornitore.

La cattura finisce in una destinazione che non esiste L+C

Cosa vedi. Una cattura risulta smistata, ma la scheda di destinazione non si aggiorna, o compare una voce orfana con una categoria mai vista.

Perché succede. Il modello ha inventato una chiave di destinazione — plausibile, ma non tua. Se il codice la scrive senza controllare, ti ritrovi dati agganciati a niente.

Come si risolve. Valida sempre quello che torna dal modello contro l'elenco canonico delle destinazioni — sono sette: task, persone, finanze, nutrizione, salute, obiettivi, memoria (Parte 4) — e tieni quell'elenco in un posto solo del codice. Se la chiave non è nell'elenco, passa alla rete di sicurezza a regole: meglio uno smistamento grezzo che un dato orfano. La nostra build fa esattamente questo, ed è il motivo per cui la cattura non fallisce mai — al massimo smista peggio.

Il patrimonio netto non torna col foglio L+C

Cosa vedi. La scheda finanze mostra un numero che non corrisponde a quello che leggi tu sul foglio di calcolo. A volte troppo alto, a volte una riga vecchia.

Perché succede. Due cause tipiche. La prima: il file ha un foglio di riepilogo e i fogli per categoria, e il modello li somma tutti — sta contando gli stessi soldi due volte. La seconda: in un foglio con lo storico giornaliero, ha pescato la riga sbagliata invece dell'ultima.

Come si risolve. Sii esplicito nelle istruzioni di estrazione: *non contare due volte gli stessi importi; da un foglio con serie storica usa solo la riga più recente*. E aggiungi al formato di risposta un campo note dove il modello può segnalare le ambiguità che trova: quando un numero non gli torna, è meglio che te lo dica lì che deciderlo in silenzio.

L'interfaccia

Pagina bianca appena carichi L+C

Cosa vedi. La dashboard non compare: pagina bianca, o un errore generico lato client. Nella console del browser c'è un errore su una proprietà letta da qualcosa che è `null`.

Perché succede. Il componente disegna prima che i dati siano arrivati. Al primo render lo stato è ancora vuoto, il codice legge un campo che non c'è, e tutto si ferma lì. In JavaScript, senza compilatore che te lo segnali prima, questo errore lo vedi così: a schermo, subito — è il costo dichiarato nella Parte 1.

Come si risolve. Un ramo di caricamento esplicito *prima* del ramo che disegna i dati: se lo stato non c'è ancora, mostra un contenitore vuoto e basta. È una riga, e va messa in ogni componente che parte da dati asincroni.

React protesta per l'orologio L+C

Cosa vedi. Nella console un errore lungo: l'HTML generato dal server non coincide con quello del client. Compare appena aggiungi l'ora, la data, o un saluto legato all'ora del giorno.

Perché succede. La pagina viene disegnata due volte: una sul server, una nel browser. Se il valore cambia tra le due — e un orologio cambia per definizione — le due versioni non coincidono, e React se ne lamenta. Ha ragione lui.

Come si risolve. Tutto ciò che dipende dall'orologio parte solo lato client: il valore inizia vuoto, si popola in un effetto dopo il montaggio, e fino a quel momento si mostra un segnaposto (`---` funziona benissimo). Il server disegna il segnaposto, il client pure, poi il client aggiorna: nessun conflitto.

L'anello di focus è sparito, senza errori L+C

Cosa vedi. Clicchi nella barra di ricerca e il bordo evidenziato che c'era ieri non compare più. Nessun errore in console, e nell'ispettore la regola CSS sembra al suo posto — ma il valore calcolato è vuoto.

Perché succede. Una variabile CSS definita a partire da sé stessa — capita quando derivi una variante di tema partendo "dallo stesso valore" — non viene risolta. Il browser la considera invalida e la scarta **in silenzio**: niente errore, niente avviso, solo l'effetto che sparisce.

Come si risolve. Una variabile non si definisce mai a partire da sé stessa. Se ti serve una variante derivata, o la ricalcoli dai valori di base, o le dai un secondo nome. E quando un effetto visivo

sparisce senza errori, il posto dove guardare è il valore *calcolato* nell'ispettore, non la regola scritta.

La colonna degli importi sembra piena di errori L+C

Cosa vedi. In una colonna di cifre, `7150` sta sopra `12.400`. Uno ha il separatore delle migliaia, l'altro no: sembra un errore di battitura, e l'occhio inciampa a ogni riga.

Perché succede. Non è un bug del codice — è una convenzione tipografica. L'italiano non raggruppa i numeri di quattro cifre: `7150` resta così, `12400` diventa `12.400`. Giusto in un testo, sbagliato in una colonna, dove i numeri vanno confrontati in verticale.

Come si risolve. Nella formattazione degli importi forza il raggruppamento anche sulle quattro cifre (`minimumGroupingDigits: 1` su `Intl.NumberFormat`). E tieni **tutta** la formattazione dei numeri in un modulo solo, così la regola vive in un posto e vale ovunque. Insieme alle cifre a larghezza fissa e all'allineamento a destra, è quello che rende le colonne leggibili.

Il pannello di dettaglio mostra la persona sbagliata L+C

Cosa vedi. Apri il dettaglio di una scheda del CRM. Nel frattempo arriva una cattura — o sposti la scheda di fascia — e il pannello ora mostra un'altra persona. Oppure si chiude da solo.

Perché succede. La selezione è agganciata alla **posizione** nella lista: "l'elemento numero 3". Ma la lista è viva — una cattura nuova entra in testa e sposta tutti gli indici di uno. Il pannello continua a guardare la posizione 3, che adesso è occupata da qualcun altro.

Come si risolve. La selezione si aggancia a un identificativo stabile — un id, o il nome — e la posizione si ricalcola a ogni render cercando quell'identificativo nella lista corrente. Vantaggio non piccolo: quando dal pannello sposti la scheda di fascia, il pannello resta agganciato e mostra subito il valore nuovo.

I dati

Spunti una cosa e dopo qualche secondo torna com'era L+C

Cosa vedi. Spunti un'abitudine, si spunta, e poco dopo — o al refresh — è di nuovo da fare. L'interfaccia ti aveva detto di sì, i dati dicono di no.

Perché succede. L'aggiornamento è ottimistico: la vista cambia subito, la scrittura parte dietro. Se la scrittura fallisce e l'errore viene inghiottito — il classico `catch` vuoto — lo schermo resta a raccontare una cosa mai salvata, finché una rilettura non lo smentisce. Sul Percorso Completo c'è

una causa in più da conoscere: un inserimento a cui manca una colonna obbligatoria fallisce solo la **prima** volta — le funzioni di upsert passano quando la riga esiste già e si rompono sulla riga nuova.

Come si risolve. Mai un gestore d'errore vuoto: come minimo si logga. E al fallimento della scrittura si rilegge lo stato vero dal server, così lo schermo torna onesto da solo — è il pattern della nostra build. Se sei sul Completo e l'errore parla di colonne, controlla che l'inserimento abbia tutti i campi obbligatori.

Aggiungi un elemento, appare, e sparisce — ma il salvataggio era andato L+C

Cosa vedi. Appena aperta la pagina aggiungi qualcosa. Compare. Un attimo dopo sparisce, anche se nel log la scrittura risulta riuscita.

Perché succede. Al montaggio la pagina fa una lettura iniziale dello stato. Se quella lettura è lenta e arriva **dopo** la tua prima modifica, la risposta — che è la fotografia di prima — sovrascrive quello che hai appena fatto. È una gara di tempi (una *race condition*), e la perde sempre l'utente veloce.

Come si risolve. Un segnalino "sporco": alla prima modifica dell'utente si alza, e da quel momento la risposta della lettura iniziale si ignora. In alternativa, la lettura iniziale si applica solo se lo stato locale è ancora intatto.

Dopo un refresh vedi il dato di prima L+C

Cosa vedi. Fai una modifica, ricarichi, e c'è il valore vecchio. Ricarichi ancora e magari compare quello nuovo. Sembra un fantasma; è una cache.

Perché succede. Next mette in cache in modo aggressivo sia le rotte che le letture `fetch`: se non gli dici il contrario, una rotta di dati può servire una risposta congelata. Sul Percorso Completo può aggiungersi un secondo strato: la edge cache del database, che sulle letture di massa può servire fotografie non aggiornate.

Come si risolve. Le rotte che servono dati dichiarano `dynamic = 'force-dynamic'`, e le letture dal client passano `cache: 'no-store'` — nella nostra build è così su tutta la linea. Se sei sul Completo e il fantasma resta, guarda la cache del database: rendere le letture più specifiche con i filtri di solito basta a non ricadere sulla stessa chiave di cache.

Solo online C

Funziona in locale, esplode dopo il deploy

Cosa vedi. In locale tutto liscio. Sul deploy, una pagina — spesso quella del calendario — va in errore a runtime, con un messaggio strano su una funzione globale che "non esiste" (classico:

qualcosa su `BigInt`).

Perché succede. In produzione il codice viene impacchettato per l'ambiente serverless, e alcune librerie che dipendono da funzioni native di Node non sopravvivono al reimpacchettamento. In locale girano sul Node vero e nessuno se ne accorge.

Come si risolve. Non combattere il bundler: cambia libreria. Per ogni formato esiste quasi sempre un parser in puro JavaScript — per il calendario, il parser di riferimento di Mozilla lo è — e con quello il problema sparisce alla radice. Regola generale: se una libreria si rompe **solo** sul deploy, sostituirla costa meno che configurarci intorno.

Le abitudini si azzerano nel cuore della notte

Cosa vedi. All'una o alle due di notte — sei ancora sveglio — le spunte di oggi spariscono. E la mattina il conteggio del giorno sembra indietro di uno.

Perché succede. Il server vive in UTC, e per lui la mezzanotte scatta quando in Italia è l'una (o le due, con l'ora legale). Ogni punto del codice che chiede "che giorno è?" al server riceve la risposta del fuso sbagliato — e nessun errore lo segnala, perché non è un errore: è un'altra mezzanotte.

Come si risolve. Una funzione sola risponde alla domanda "che giorno è oggi", usa il fuso dichiarato in `USER_TIMEZONE` , e viene chiamata **ovunque** serve una data: chiave di archiviazione, riga del log giornaliero, azzeramenti. Se la domanda è sparsa nel codice, prima o poi un punto risponde in UTC.

Il bot non risponde

Cosa vedi. Scrivi al bot e non succede niente. Nessuna conferma, nessuna cattura, nessun errore da nessuna parte — il silenzio.

Perché succede. Le cause si somigliano tutte: il webhook non è registrato, è registrato su un indirizzo vecchio, il segreto non coincide, oppure il tuo endpoint sta rispondendo errori. E ricorda che un bot può puntare a **un** indirizzo solo: se lo stesso bot lo usi anche per le prove, i messaggi vanno dove punta adesso, non dove pensi tu.

Come si risolve. Prima mossa, sempre: interroga `getWebhookInfo` (il comando è nella Parte 7.4). Ti dice a che indirizzo punta il bot e qual è stato l'ultimo errore — che è esattamente l'informazione da incollare all'agente, intera. Da lì la diagnosi è corta.

La stessa nota archiviata cinque volte

Cosa vedi. Una nota vocale, cinque catture identiche in fila.

Perché succede. Il tuo endpoint ha risposto con un errore, e Telegram — che è fatto così — ha riprovato a consegnare lo stesso messaggio. A ogni tentativo, una scrittura in più.

Come si risolve. Il webhook risponde **sempre** 200, anche quando dentro qualcosa va storto: l'errore si gestisce e si logga all'interno, la risposta verso Telegram resta buona. Se vuoi la cintura oltre alle bretelle, tieni traccia degli identificativi dei messaggi già processati e scarta i doppioni.

La nota vocale arriva, la trascrizione è vuota

Cosa vedi. Il messaggio vocale viene ricevuto, la pipeline gira, e il testo trascritto è una stringa vuota. Nessun errore.

Perché succede. L'audio di Telegram arriva in formato OGG. Il servizio di trascrizione lo gestisce, ma solo se il file gli viene passato con il tipo di contenuto giusto: con un MIME sbagliato o un nome file senza estensione, invece di protestare restituisce il nulla.

Come si risolve. Nel passaggio del file alla trascrizione, dichiara esplicitamente formato ed estensione corretti. È una riga, e il sintomo — vuoto senza errore — è così anonimo che senza saperlo prima ci si perde un'ora.

Hai cambiato una variabile e non è cambiato niente

Cosa vedi. Aggiorni una variabile d'ambiente nel pannello, ricarichi l'app, e si comporta esattamente come prima.

Perché succede. Le variabili vengono lette al momento della build, non a ogni richiesta. L'app che sta girando è stata costruita con i valori vecchi, e nessuno le ha detto di ricostruirsi.

Come si risolve. Dopo ogni modifica alle variabili, un nuovo deploy. Tutto qui — ma se non lo sai, è una mezz'ora persa a fissare un pannello.

"Chiave non valida", ma la chiave è giusta

Cosa vedi. Hai incollato la chiave corretta — ricontrollata tre volte — e il servizio continua a rifiutarla.

Perché succede. Le chiavi private degli account di servizio contengono degli a-capo. Nel viaggio tra il file JSON e il campo del pannello, gli a-capo si perdono o diventano `\n` letterali, e la chiave — pur "giusta" — non è più leggibile.

Come si risolve. Reincollala con attenzione, verificando che gli a-capo siano sopravvissuti, oppure fai ripristinare al codice gli `\n` letterali al momento della lettura. È la causa numero uno di questo messaggio, come detto nella Parte 7.3 — vale la pena escluderla per prima.

Parte 9 • Dove portarlo dopo

Il sistema è vivo e lo stai usando. Da qui in poi non c'è un ordine giusto — c'è il tuo.

Questa lista è ordinata per **utilità reale**, cioè per quanto cambia le tue giornate, non per quanto è difficile. Alcune cose facili stanno in fondo perché contano poco; una o due cose impegnative stanno in alto perché contano molto.

Il tempo indicato è una stima ragionevole per chi ha appena finito la build e ha ancora il progetto in testa. Se ci torni tra due mesi, raddoppia.

Una regola prima di cominciare: aggiungi una cosa alla volta e usala per una settimana prima di aggiungerne un'altra. Un sistema personale muore per accumulo, non per mancanza di funzioni. Quasi tutto quello che aggiungi ha un costo di attenzione, e le cose che non guardi mai sono rumore che ti fa guardare meno anche il resto.

1 • La revisione settimanale automatica

Cosa aggiunge. Il venerdì sera il sistema ti scrive: cosa hai chiuso, cosa è slittato per la terza settimana di fila, quali persone non senti da troppo, dove sono finiti i soldi. È il salto da "un posto dove archivio le cose" a "una cosa che mi fa notare gli schemi" — ed è il momento in cui il sistema comincia a restituirti più di quanto gli dai.

Quanto costa. Un paio d'ore. È lo stesso stampo del briefing del mattino: un cron, una query più larga, un prompt più ambizioso.  — sul Locale puoi farla partire a mano da un pulsante, e funziona quasi altrettanto bene.

2 • Il backup automatico

Cosa aggiunge. Dormire tranquillo. Un cron settimanale che salva il dump in un repository privato ti dà anche lo storico: puoi vedere com'era il sistema tre mesi fa. La rotta è una sua, sotto `/api/cron/`, protetta col `CRON_SECRET` — l'export della 7.6 vuole la tua sessione e un cron non ce l'ha — ma la funzione che raccoglie le tabelle è la stessa.

Quanto costa. Un'ora, se l'endpoint di export l'hai già fatto nella Parte 7. 

3 • Annulla e modifica in blocco nel CRM

Cosa aggiunge. Non è una funzione, è ergonomia — ed è il motivo per cui continuerai a usare il CRM invece di lasciarlo marcire. Selezioni dieci task, li chiudi in un colpo, e se hai sbagliato un avviso in basso ti fa tornare indietro. La differenza si sente nella sistemata del venerdì.

Quanto costa. Due o tre ore, quasi tutte di interfaccia. La parte delicata è che l'annullamento funzioni anche dopo che la pagina ha già ricaricato i dati. [L+C](#)

4 • Sottotask e progetti

Cosa aggiunge. Il momento in cui ti serve è riconoscibile: quando cominci a scrivere task che si chiamano "Video — montaggio parte 2". Una colonna che punta al task padre e le liste smettono di essere piatte.

Quanto costa. Due o tre ore. La struttura dei dati è mezz'ora; il resto è farla vedere bene senza costruire un gestore di progetti in miniatura. Resisti alla tentazione: un livello di annidamento basta.

[L+C](#)

5 • Il diario vocale lungo

Cosa aggiunge. Una cosa è dettare "ricordati di chiamare Marco", un'altra è parlare tre minuti a fine giornata. Il secondo caso non deve finire tra i task: va in un posto suo, e diventa il materiale più prezioso che la memoria ha da darti dopo qualche mese.

Quanto costa. Un'ora o due. La pipeline c'è già: cambia dove finisce il testo e come lo indicizzi. La scelta vera è di prodotto — salvi la trascrizione intera, un riassunto, o entrambi. Consiglio: entrambi, cerchi sul riassunto e leggi l'originale. [C](#)

6 • La cattura senza aprire niente

Cosa aggiunge. Toglie gli ultimi tre secondi di attrito. Una scorciatoia iOS legata al tasto azione o a Siri, che detta e manda al tuo endpoint. Sembra poco: è la differenza tra catturare cinque cose al giorno e catturarne quindici.

Quanto costa. Un'ora, quasi tutta a litigare con l'editor delle scorciatoie. Sul Completo non serve niente lato server: l'endpoint accetta già l'intestazione segreta. [C](#)

Sul Locale, invece, fermati. Il cancello e il segreto delle API sono roba del Percorso Completo: nel tuo `.env.local` `API_SECRET` non c'è, e le tue rotte di scrittura sono aperte perché finora l'unico a poterle chiamare eri tu, da localhost. Prima di esporre qualsiasi cosa fuori da localhost — anche solo alla rete di casa — devi costruire il cancello della Parte 3 (il prompt è A8) e darti un segreto. Una dashboard con le rotte di scrittura aperte non si mette in rete, nemmeno in casa: la rete di casa ha ospiti, telefoni altrui e una televisione che parla con mezzo mondo.

7 • Passare al Completo [L → C](#)

Cosa aggiunge. Quello che al Locale manca per come è fatto: il sistema sveglia quando il tuo computer dorme, la cattura da Telegram mentre sei in fila alla cassa, il briefing che ti cerca la

mattina. E la memoria per somiglianza, che è il pezzo tecnico della salita: sul Locale funziona passando tutto al modello — preciso e semplice, finché "tutto" sta nel contesto. Quando le catture diventano migliaia, le domande cominciano a costare e a peggiorare; gli embedding risolvono selezionando i venti pezzi rilevanti invece di spedire l'archivio.

Come si fa. In tre mosse, in quest'ordine.

Prima **crei le tabelle**: è la migrazione di A6, con il passaggio 🖐 che la applica davvero — SQL Editor del pannello, incolla, esegui. Finché le tabelle non ci sono, non c'è dove travasare.

Poi **porti lo store sul database e travasi**: prima di lanciare il prompt servono lo store su Supabase (A7) — è lui che d'ora in poi legge e scrive le tabelle al posto del file — e lo strato di memoria (A15), perché è la rotta che genera gli embedding:

PROMPT · DA INCOLLARE IN CLAUDE CODE

Ho una dashboard PersonalOS sul Percorso Locale, con i dati in `data/personalos.json`, e ho appena creato le tabelle su Supabase. Scrivi uno script una-tantum che legge quel file e travasa nelle tabelle: profilo, task, persone, catture, log giornalieri. Rispetta gli identificativi esistenti dove ci sono, e riporta a schermo quante righe ha scritto per ogni tabella. Poi rigenera gli embedding di tutte le voci di memoria con la rotta dello strato di memoria, in blocchi, mostrando l'avanzamento. Il file locale non si cancella e non si modifica: resta lì come backup.

Verifica: i conteggi per tabella che lo script ti stampa coincidono con quelli che leggi nel file. Poi una domanda alla memoria su una cosa vecchia: se torna con la fonte citata, gli embedding ci sono tutti.

Infine **il resto della Parte 7**: cancello (se non l'hai ancora costruito, A8), variabili, deploy, webhook.

Quanto costa. Mezza giornata per la memoria e il travaso, più il tempo della Parte 7 se parti da zero. Il segnale per farlo è misurabile: quando una domanda ci mette più di dieci secondi, quando il modello comincia a saltare cose che sai di avergli dato, o — più semplicemente — quando ti accorgi che il pensiero migliore della giornata l'hai perso perché eri per strada.

8 · La vista in sola lettura da condividere

Cosa aggiunge. Una versione con una password diversa che mostra solo alcune schede — utile se lavori con qualcuno, o se in casa i piani si incastrano.

Quanto costa. Tre o quattro ore, e la parte lunga non è tecnica: è decidere cosa è condivisibile. Fallo per elenco di cose permesse, mai per elenco di cose escluse. La differenza è che con il primo

la scheda che aggiungi tra sei mesi è invisibile finché non decidi tu; con il secondo è pubblica finché non te ne ricordi. **C**

9 • Più utenti

Cosa aggiunge. Solo una cosa: la possibilità di darlo ad altri. Se non è quello che vuoi fare, non ha alcun valore per te.

Quanto costa. Diverse serate, e cambia la natura del progetto. Ogni query prende un filtro sull'utente, servono le regole di accesso a livello di riga, un bot per persona o una mappa tra chat e utenti, e all'improvviso hai dei dati altrui di cui sei responsabile — che è una faccenda seria, non un dettaglio tecnico. L'architettura non cambia; cambia il tuo ruolo. **C**

10 • Lo smistamento con un modello locale

Cosa aggiunge. Che le tue frasi non escono più dalla tua macchina. Se è questo che ti tiene lontano dal sistema, è l'unica voce di questa lista che conta.

Quanto costa. Una serata per farlo funzionare, più il tempo di provarlo davvero: prendi cinquanta frasi tue reali e confronta le destinazioni con quelle di prima. Sulle frasi ambigue aspettati un calo. Per lo smistamento potrebbe bastarti; per le risposte sulla memoria la differenza si sente di più. Il resto del sistema non cambia: è un fornitore diverso dietro la stessa funzione. **L**

Una nota finale su questa lista. Nessuno di questi dieci punti è obbligatorio, e un sistema che fa bene le cose della build base è meglio di uno che fa dieci cose a metà. Il segnale per aggiungere qualcosa è sempre lo stesso: **te ne sei accorto usandolo**. Se stai scegliendo dalla lista perché ti sembra che manchi qualcosa, non è ancora il momento.

Appendice A · Tutti i prompt in sequenza

Ogni prompt del documento, nell'ordine in cui si costruisce — schede comprese. Dopo la prima lettura puoi lavorare da qui: scendi la lista, salta quello che non è del tuo percorso, incolla, verifica, avanti. Quello che qui non c'è sono le spiegazioni: i *perché* stanno nel corpo, e le trappole che ogni prompt evita le capisci solo lì.

Come si leggono i marcatori: **L** solo Percorso Locale · **C** solo Percorso Completo · **L+C** entrambi · **L → C** solo per chi sale dal Locale al Completo. Se segui il Locale, i blocchi **C** non ti riguardano: saltali senza leggerli. E viceversa. 🖐️ segnala un passaggio che devi fare tu, fuori da Claude Code.

L'ordine qui è quello **di costruzione**, che in due punti differisce dall'ordine delle parti: il mockup si disegna prima di tutto, ma si porta dentro al progetto solo dopo averlo creato; e il cancello si costruisce appena esistono i dati — che è anche il punto in cui sta nel corpo, nella Parte 3.

A1 · Il mockup **L+C**

Produce un solo file HTML con le quattro schermate, pronto da portare nel progetto. Da incollare in Claude Design o in qualsiasi strumento che generi HTML — i requisiti sono quelli della Parte 2. Se non usi uno strumento di design, questo prompt puoi darlo direttamente al tuo agente: un file HTML lo sa scrivere anche lui.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Genera il mockup di una dashboard personale in un solo file HTML, con il CSS dentro, senza dipendenze esterne e senza immagini remote; le icone in SVG, scritte nel file. I colori come variabili in cima al foglio di stile. Un contenitore per la barra in alto e uno per la griglia. Quattro schermate: Home (saluto e ora, chi sono e il focus di oggi, abitudini con percentuale, calendario della settimana, blocchi, polso finanziario, nutrizione, andamento fisico degli ultimi trenta giorni, obiettivi di settimana e mese), CRM con gli elementi da fare divisi per fascia di urgenza, ognuno con la persona collegata, Finanze, Review della settimana. Una sezione per ogni scheda, ognuna con un identificativo parlante. Dati di esempio scritti a mano.

Verifica: aprilo nel browser **senza rete**. Se si vede tutto lo stesso, è autosufficiente davvero.

A2 · Il progetto **L+C**

Produce il progetto Next.js vuoto in cui vive tutto il resto.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea un progetto Next.js (noi siamo sulla serie 16) con App Router, in JavaScript, senza Tailwind, senza ESLint e senza cartella `src`.

Verifica: `npm run dev` parte senza errori e la pagina di default risponde nel browser.

A3 · I font L+C

Produce il caricamento reale dei caratteri del mockup, così quello che vedi tu è quello che si vede ovunque.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggancia i font con `next/font/google` ed esponili come variabili CSS, poi usale nel foglio di stile.

Verifica: ispeziona un testo nel browser: il carattere calcolato dev'essere quello del mockup, non quello di sistema. La controprova arriva col porting: fatto coi font sbagliati, cambia faccia a tutto.

A4 · Il porting del mockup L+C

Produce i componenti React a partire dall'HTML di A1: barra in alto, griglia, una schermata per file.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Ho un mockup HTML in `[percorso]`. Portalo in componenti React dentro `components/`: un componente per la barra in alto, uno per la griglia contenitore, e uno per ogni schermata. Le schede restano sezioni dentro la schermata a cui appartengono. Prendi il CSS così com'è e mettilo in `app/globals.css` senza riscriverlo. Per ora usa dati scritti a mano dentro i componenti: i dati veri li colleghiamo dopo.

Verifica: le quattro schermate rendono come il mockup, con i dati finti al loro posto.

A5 · Lo store su file L

Produce `lib/store.js` e i due file dati: il seed intatto e la copia di lavoro.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea `lib/store.js` con quattro funzioni: leggere lo stato, scriverlo, aggiornarlo con una funzione, e riportarlo alle condizioni iniziali. Aggiungi una funzione che legge i log giornalieri di un intervallo di date. Lo stato vive in `data/personalos.json` e contiene anche il **profilo** — nome, ruolo, città, focus del giorno, elenco delle abitudini, obiettivo calorico giornaliero: sono dati di configurazione e la loro casa è lì. Al primo avvio, se il file non c'è, va copiato da `data/seed.json`, che non viene mai modificato e che porta già dentro il profilo di partenza. Aggiungi `data/personalos.json` al `.gitignore`.

Verifica: cancella `data/personalos.json` e ricarica la pagina: si rigenera da solo, identico al seed. Quel gesto è il tuo tasto di annullamento — controlla subito che funzioni.

A6 · Le tabelle su Supabase

Produce la migrazione con le sette tabelle, la funzione di ricerca vettoriale e la protezione a livello di riga. 🖐 Prima serve il progetto su `supabase.com`, con i tre valori dal pannello e l'estensione `vector` attiva (Parte 3).

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea una migrazione con queste tabelle: **profilo** (una riga sola, di configurazione: nome, ruolo, città, focus del giorno, elenco delle abitudini, obiettivo calorico giornaliero), **catture** (testo grezzo, provenienza, classificazione, dove è stata smistata, via di classificazione), **task** (titolo, urgenza, priorità, tag, scadenza, persona collegata), **persone** (nome, organizzazione, tipo, metadati), **log giornalieri** (una riga per data, con un campo JSON per abitudini, pasti, obiettivi e finanze), **memoria** (testo, provenienza, e l'embedding come `vector(1536)`), **registro** (chi ha fatto cosa e quando). Su ogni tabella una colonna `user_id`, valorizzata dalla variabile d'ambiente `USER_ID`. Aggiungi una funzione SQL `match_memoria` che riceve un embedding e un numero, ordina per distanza coseno e restituisce le voci più vicine, più un indice HNSW sulla colonna dell'embedding. Attiva la protezione a livello di riga su tutte, in negazione totale: solo la chiave di servizio passa.

Verifica: 🖐 prima applicala — SQL Editor del pannello Supabase, incolla il file di migrazione, esegui; oppure `supabase login`, `supabase link`, `supabase db push`. Poi: le sette tabelle compaiono nel pannello, `match_memoria` compare tra le funzioni, e una lettura fatta con la chiave pubblica torna vuota — se torna dei dati, le regole di riga non stanno proteggendo.

A7 · Lo store sul database

Produce la versione di `lib/store.js` che parla con Supabase invece che col file — stessa interfaccia, fondo diverso. È il prompt della Parte 3.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea `lib/store.js` come unico punto di accesso ai dati, sopra Supabase. Usa l'indirizzo del progetto e la chiave di servizio dalle variabili d'ambiente, e assicurati che il modulo venga importato solo da codice che gira sul server, mai da componenti client. Esponi funzioni per: leggere e aggiornare il profilo; leggere e scrivere le catture, i task, le persone, le voci di memoria e il registro; leggere e aggiornare il log giornaliero di una data, creando la riga se non esiste; e leggere i log giornalieri di un intervallo di date. Nessuna rotta e nessun componente tocca il database direttamente: tutti chiedono a questo modulo. Mantieni le stesse firme del modulo su file del Percorso Locale, così le rotte restano identiche nei due percorsi.

Verifica: fai passare una scrittura di prova dal modulo e guardala comparire nel pannello di Supabase; una lettura con la chiave pubblica dal browser deve continuare a tornare vuota.

A8 · Il cancello

Produce il middleware, la pagina di login e le rotte di autenticazione. Si fa **adesso**, appena esistono i dati — non quando arrivi al deploy. È il prompt completo della Parte 3.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi un cancello a password singola all'app.

- middleware che protegge tutte le rotte tranne: `/login`, `/api/auth/*`, `/api/telegram/webhook` e `/api/cron/*`
- cookie di sessione firmato con HMAC usando `AUTH_SECRET`, `httpOnly`, `secure`, `sameSite lax`, scadenza 30 giorni
- pagina `/login` con un solo campo password, che invia a `/api/auth/login`
- `/api/auth/login` confronta con `DASHBOARD_PASSWORD` usando un confronto a tempo costante e imposta il cookie
- `/api/auth/logout` cancella il cookie
- le rotte API accettano in alternativa l'intestazione `x-api-secret` confrontata con `API_SECRET`, anche questo con un confronto a tempo costante, per l'accesso da script
- intestazione `X-Robots-Tag`: `noindex` su tutte le risposte

Poi mostrami l'elenco esatto delle rotte escluse dal middleware e come ognuna si protegge da sola.

Verifica: l'agente ti mostra l'elenco delle esclusioni — leggilo tu, riga per riga, chiedendoti chi può chiamare ognuna e con cosa. Poi apri il sito in una finestra anonima: deve chiederti la password.

A9 · Lo smistatore L+C

Produce `lib/classify.js`: la funzione che decide dove va ogni frase, con la rete di sicurezza a regole.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea `lib/classify.js`. Riceve un testo e restituisce `{ destinazione, titolo, persona, urgenza }`. Le destinazioni possibili sono sette: task, persone, finanze, nutrizione, salute, obiettivi, memoria. L'urgenza è una tra oggi, settimana, più avanti, con predefinito "oggi" quando il modello non si sbilancia; "in ritardo" non è mai un valore in ingresso — in ritardo ci si finisce, non ci si nasce. Usa Claude con un prompt di sistema che elenca le destinazioni possibili e chiede di rispondere solo in JSON. Il nome del modello leggilo dalla variabile d'ambiente `ANTHROPIC_MODEL`, lasciando nel codice un valore predefinito per quando manca. Non attivare il ragionamento esteso su queste chiamate, e se il tuo setup lo accende di default qui va spento: non serve e allunga l'attesa. Se la chiamata fallisce o la chiave manca, ripieghi su delle regole basate su parole chiave, così la funzione non si pianta mai. Ogni cattura salva anche **come** è stata classificata — modello primario, riserva, o regole — e la dashboard lo mostra in modo discreto sulle catture recenti.

Verifica: prova cinque frasi diverse — una spesa, una persona da richiamare, un pasto, un pensiero, un task — e guarda destinazioni e urgenze. Poi toglie la chiave API dall'ambiente e riprova: deve rispondere lo stesso, via regole, e la cattura deve risultare **marcata "regole"** nella dashboard. Se risponde ma non è marcata, la rete di sicurezza sta lavorando in silenzio — ed è il guasto della Parte 8 "Smista peggio da giorni".

A10 · La rotta di cattura L+C

Produce l'endpoint che riceve un testo e lo scrive nei tre posti: catture, memoria, scheda di destinazione.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea la rotta di cattura. Riceve un testo, lo fa classificare, scrive nelle catture, aggiunge una voce in memoria, e se la destinazione ha una scheda dedicata aggiorna anche quella. Restituisce la destinazione, così l'interfaccia può mostrarla.

Verifica: una chiamata con `curl` alla rotta risponde con la destinazione, e la cattura compare nei dati.

A11 · La barra di cattura L+C

Produce la barra di cattura fissa in fondo alla dashboard, con il microfono del browser e i quattro stati.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi la barra di cattura fissa in fondo, con un pulsante microfono che usa il riconoscimento vocale del browser, un campo di testo, e i quattro stati. Invio manda alla rotta di cattura. A operazione finita mostra dove è stata smistata la frase, poi torna a riposo dopo un paio di secondi.

Verifica: scrivi una frase e premi invio: ti dice dove l'ha messa, e la scheda giusta si è aggiornata. Il microfono provalo in Chrome.

A12 · Il webhook Telegram C

Produce la rotta che riceve i messaggi del bot, trascrive i vocali e li passa alla stessa pipeline della dashboard. Prima servono token, segreto e ID utente (Parte 4). La **registrazione** del webhook si fa dopo il deploy, con il comando della Parte 7.4.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea la rotta del webhook Telegram. Verifica l'intestazione col segreto, verifica che il mittente sia il tuo identificativo, e scarta tutto il resto. Se il messaggio è vocale, scarica l'audio e trascrivilo con Whisper di OpenAI. Poi passa il testo alla stessa pipeline di classificazione che usa la dashboard. Rispondi su Telegram con la conferma e dei pulsanti per correggere con un tocco l'urgenza che il classificatore ha assegnato. La stessa rotta gestisce anche gli aggiornamenti di tipo `callback_query`, quelli che arrivano dai pulsanti: lì il mittente sta in `callback_query.from.id` e non nel messaggio, si risponde sempre con `answerCallbackQuery` — altrimenti il pulsante resta a girare sul telefono — e si aggiorna l'elemento già salvato.

Verifica: dopo deploy e registrazione — un messaggio di testo al bot torna con la conferma; una nota vocale viene trascritta e smistata. Se il bot tace, Parte 8: "Il bot non risponde".

A13 · Le schede sui dati veri L+C

Produce il collegamento tra le schermate di A4 e i dati veri, con le regole della Parte 5. Si comincia fissando le regole comuni:

PROMPT · DA INCOLLARE IN CLAUDE CODE

Collega le schede ai dati veri, una schermata alla volta. Ogni scheda legge e scrive passando dalle rotte API, mai toccando i dati direttamente. Nessuna scheda chiama il modello al caricamento della pagina. I numeri si allineano a destra e usano le cifre a larghezza fissa. Il colore segue la metrica, non il segno: verde per ciò che va nella direzione giusta, rosso per il contrario, arancione per ciò che scade. Ogni gesto aggiorna subito la vista e salva dietro; se il salvataggio fallisce, rileggi lo stato vero invece di lasciare a schermo una cosa non salvata.

Verifica: apri il pannello dei consumi API dopo dieci minuti di navigazione: deve essere fermo — se si muove, qualcosa chiama il modello al caricamento.

Poi le schede, **una alla volta**, nell'ordine qui sotto, che è quello della Parte 5: dalla più facile alla più difficile. Qui trovi il prompt e la verifica di ognuna; le spiegazioni — cioè le trappole che ogni prompt sta evitando — stanno nella sezione corrispondente del corpo, e la prima volta vanno lette. Attenzione ai bivi: Abitudini (5.3), Obiettivi (5.7) e Polso finanziario (5.8) hanno varianti diverse per i due percorsi, e il Polso sul Completo richiede prima 🖐️ l'account di servizio Google (procedura nella 5.8).

A13.1 · Operator + Session L+C

Produce le due schede del mattino: chi sei, e le tre cose che contano oggi.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci le schede Operator e Session. Operator è quasi statica: legge dal profilo nome, ruolo, città e focus del giorno, e calcola la striscia dai log giornalieri delle abitudini, contando all'indietro i giorni consecutivi con almeno un'abitudine completata. Session mostra saluto in base all'ora, orologio, data per esteso, e sotto i tre task principali di oggi letti dal CRM: filtra gli elementi nelle fasce "in ritardo" e "oggi", ordinali per fascia, poi per temperatura, poi per posizione manuale, e mostra i primi tre con titolo e persona collegata. Cliccando un task si apre la schermata CRM con il pannello di dettaglio di quell'elemento già aperto. Se i task sono meno di tre mostra quelli che ci sono; se non ce ne sono, scrivilo. L'orologio si avvia solo lato client, per evitare il conflitto tra l'ora del server e quella del browser.

Verifica: nome, ruolo e focus sono i tuoi, letti dal profilo, non quelli scritti a mano nel mockup; l'orologio gira e la console del browser è pulita — nessun errore di hydration. I tre task di Session li ricontrolli a CRM finito: quello che sta in cima alla fascia più urgente deve comparire qui.

A13.2 · Calendario L+C

Produce la rotta che legge il feed iCal e la striscia dei sette giorni. 🖐 Prima serve l'indirizzo segreto del calendario, preso da te dalle impostazioni di Google Calendar.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la scheda Calendario. Una rotta API legge l'indirizzo iCal dalla variabile d'ambiente `GOOGLE_CALENDAR_ICAL_URL`, scarica il feed, lo interpreta con un parser iCal in puro JavaScript ed espande gli eventi ricorrenti su una finestra di quattordici giorni. Tieni il risultato interpretato in una cache in memoria per cinque minuti; la risposta HTTP porta l'intestazione che vieta al browser di conservarla. La scheda mostra la striscia dei sette giorni della settimana corrente: cliccando un giorno si vedono i suoi appuntamenti con orario, titolo e calendario di provenienza. Nel giorno di oggi disegna un marcatore all'ora corrente e porta la vista lì. Tutti gli orari si mostrano nel fuso dell'utente.

Verifica: apri la scheda e cerca un tuo impegno **ricorrente** — la riunione del lunedì, il corso del giovedì: se ci sono solo gli eventi singoli, le regole di ripetizione non vengono espanse. Poi sposta un evento su Google e ricarica: entro cinque minuti la scheda lo segue.

A13.3 · Abitudini L+C

Produce la scheda delle abitudini del giorno, con l'anello di completamento e la funzione unica della data.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la scheda Abitudini. L'elenco delle abitudini si legge dal profilo — è lì che si configurano, non scritte a mano nel codice — e ogni abitudine è di uno di due tipi: spunta, oppure contatore con obiettivo. Lo stato del giorno vive nel log giornaliero, con la data calcolata da un'unica funzione che usa il fuso di `USER_TIMEZONE` — mai la data del server. Il clic spunta, incrementa, o azzerava un contatore pieno; la vista si aggiorna subito e la scrittura passa dalla rotta API. In alto un anello con la percentuale di completamento: le spunte valgono piene, i contatori in proporzione. A ogni nuovo giorno la scheda riparte vuota; i giorni passati restano nel log.

Verifica: spunta un'abitudine e ricarica la pagina — resta spuntata. Togli o aggiungi un'abitudine nel profilo e ricarica: la scheda cambia senza che nessuno abbia toccato il codice. Poi apri il file dei dati e controlla che la spunta sia finita nella riga del **giorno locale**, non in quella di ieri o di domani.

A13.3-bis · La sincronizzazione delle Abitudini C

Produce il pezzo che manca sul Completo: stessa spunta dal telefono e dal computer, senza che una risposta vecchia ti cancelli il clic.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi la sincronizzazione alla scheda Abitudini: cache in localStorage per il rendering immediato, scrittura sul database a ogni clic, e all'apertura una lettura degli ultimi trenta giorni da fondere con la cache. Dal primo gesto dell'utente in poi, marca lo stato come modificato e ignora le risposte di letture partite prima: una risposta vecchia non deve mai sovrascrivere un clic appena fatto.

Verifica: spunta un'abitudine dal telefono e guarda il computer — dopo un ricaricamento è spunta anche lì. Poi la prova cattiva: ricarica la pagina e clicca **subito**, prima che la lettura d'apertura sia tornata. La spunta deve restare; se dopo un secondo torna indietro, la risposta vecchia sta ancora sovrascrivendo il tuo clic.

A13.4 · CRM L+C

Produce la scheda più grande: le tre viste, il Kanban con il riordino, il pannello di modifica. Costruiscila in due passaggi, e verifica il primo prima di passare al secondo.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la schermata CRM. Ogni elemento ha identificativo, titolo, nota, fascia (in ritardo, oggi, questa settimana, più avanti), temperatura (caldo, tiepido, freddo), persona collegata, tag, posizione, data di creazione e di completamento. Tre viste commutabili da un selettore, con la scelta salvata nel browser e ripristinata all'apertura. Kanban: quattro colonne per fascia, trascinamento tra colonne per cambiare fascia e dentro la colonna per riordinare; al rilascio si riscrivono le posizioni della fascia toccata. Per persona: gli stessi elementi raggruppati per persona, senza-persona in fondo. Cliccando un elemento si apre un pannello laterale per modificare tutti i campi, completare o eliminare; il pannello traccia l'elemento tramite l'identificativo, mai tramite la posizione in lista. I nuovi elementi — anche quelli in arrivo dalla cattura — entrano in testa alla fascia che corrisponde all'urgenza assegnata dal classificatore. Completare non cancella: l'elemento resta nei dati con la data di completamento, e l'evento finisce nel registro.

E poi, come passaggio separato:

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi la ricerca in linguaggio naturale al CRM. Una rotta riceve la domanda, passa al modello la lista compatta degli elementi aperti (identificativo, titolo, fascia, persona, tag) con l'istruzione di restituire solo un JSON con gli identificativi pertinenti, e risponde con quelli. La casella di ricerca parte solo alla pressione di Invio, mai a ogni tasto. Scarta gli identificativi che non esistono nella lista. Se la chiamata fallisce o la chiave manca, la casella ripiega su un filtro testuale semplice sugli stessi campi, senza errori a schermo.

Verifica: trascina una carta in un'altra colonna, ricarica: è rimasta lì, e anche l'ordine dentro la colonna è quello che le hai dato. Apri il pannello di un elemento e, con il pannello aperto, manda una cattura dalla barra: il pannello deve continuare a mostrare lo stesso elemento. Cambia vista, ricarica: la vista è quella che avevi scelto. Poi togli la chiave API e fai una ricerca: risponde lo stesso, in modo più stupido, senza errori a schermo.

A13.5 · Nutrizione L+C

Produce la scheda dei pasti: stima dal testo, formula in un senso, modello nell'altro.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la scheda Nutrizione. I pasti del giorno vivono nel log giornaliero, ognuno con orario, nome, calorie, proteine, carboidrati, grassi e un contrassegno "stimato". Una rotta di stima riceve la descrizione testuale di un pasto e restituisce, tramite il modello, calorie e macro in solo JSON. Una rotta di redistribuzione riceve nome del pasto e calorie di destinazione e restituisce macro coerenti. Nella scheda: campo per descrivere un pasto, totale del giorno contro l'obiettivo calorico letto dal profilo, barre dei macro, lista dei pasti. Cliccando un pasto si apre la modifica diretta: cambiando un macro le calorie si ricalcolano all'istante con la formula $4 \cdot p + 4 \cdot c + 9 \cdot f$, senza chiamate; cambiando le calorie parte, solo a digitazione conclusa, la rotta di redistribuzione. I valori stimati sono marcati come tali finché non li correggi. La data del giorno arriva dalla funzione unica col fuso dell'utente; a ogni nuovo giorno la scheda riparte vuota e i giorni passati restano nel log.

Verifica: scrivi "petto di pollo con riso e una mela" — tornano quattro numeri plausibili, marcati come stima. Cambia le proteine a mano: le calorie si muovono all'istante e il pannello dei consumi resta fermo. Cambia le calorie: parte una chiamata sola, a digitazione finita, e la stima sparisce dai valori che hai corretto tu.

A13.6 · Salute L+C

Produce la tabella dei trenta giorni, con le medie calcolate solo sui giorni registrati.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la scheda Salute. Legge gli ultimi trenta giorni dal log giornaliero e mostra una tabella con una riga per ogni giorno che ha almeno un pasto registrato: data, calorie totali, proteine, carboidrati, grassi, numero di pasti. Cliccando una riga si espande l'elenco dei pasti di quel giorno con orario e valori; un secondo clic la richiude. In testa, le medie di calorie e macro calcolate esclusivamente sui giorni registrati — i giorni senza dati non entrano nel calcolo — con accanto l'indicazione di quanti sono. Nessuna chiamata al modello, nessuna scrittura: solo lettura e aggregazione.

Verifica: le medie in testa cambiano se aggiungi un pasto, e la dicitura dice su quanti giorni sta calcolando. Aggiungi a mano un giorno vuoto nei dati: la media non si muove di una virgola. Clicca una riga e si apre nei pasti di quel giorno; ricliccala e si richiude.

A13.7 · Obiettivi L+C

Produce le due liste che non si azzerano mai da sole. La parentesi finale riguarda solo il Completo.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci la scheda Obiettivi: due sezioni, "Questa settimana" e "Questo mese". Ogni voce ha nome, stato fatto/da fare e un progresso facoltativo. In fondo a ogni sezione un campo per aggiungere una voce con Invio; su ogni voce, al passaggio, una X per rimuoverla. Spuntare segna fatto e la voce resta visibile. Gli obiettivi non si azzerano mai da soli: nessuna logica legata al cambio di settimana o di mese. [Percorso Completo: le due liste si salvano nel log giornaliero su un'unica riga con data fissa convenzionale 2000-01-01, che non viene mai fatta scadere; documenta questa scelta nel CLAUDE.md del progetto.]

Verifica: aggiungi un obiettivo, spuntalo, ricarica: è ancora lì, spuntato. La prova vera è quella del tempo — sposta la data del sistema alla settimana prossima e ricarica: le liste devono essere esattamente quelle di prima. Se si sono svuotate, qualcosa le ha agganciate al periodo.

A13.8 · Polso finanziario L

Produce la lettura del foglio da un file sul tuo disco, con l'estrazione legata al solo pulsante.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi il Polso finanziario. Una rotta legge il file del foglio di calcolo dal percorso nella variabile `FINANCE_FILE_PATH` (xlsx o csv), trasforma ogni foglio in una griglia di testo e manda tutto al modello con l'istruzione: estrai patrimonio netto, valuta, data di riferimento e categorie con nome, valore e tipo (liquidità, investito, debito); evita i doppi conteggi tra fogli di riepilogo e fogli di dettaglio; dalle tabelle con date usa solo la riga più recente; aggiungi un campo note per le ambiguità; rispondi solo in JSON. Valida il JSON e salva un'istantanea con la data. La scheda mostra patrimonio, delta rispetto alle istantanee precedenti e categorie, leggendo sempre l'ultima istantanea salvata; l'estrazione parte esclusivamente dal pulsante di aggiornamento, mai al caricamento della pagina. Accanto al pulsante, l'ora dell'ultima estrazione e il campo note se non è vuoto.

Verifica: premi il pulsante e confronta il patrimonio con quello che leggi tu sul foglio, a mano. Poi ricarica la pagina tre volte guardando il pannello dei consumi: deve restare fermo — se si muove, l'estrazione è agganciata al caricamento. E leggi il campo note: le prime volte è lì che sta la spiegazione dei numeri che non tornano.

A13.8-bis · Polso finanziario da Google Sheets

Produce la stessa scheda, ma leggendo il foglio vero. 🖐 Prima serve l'account di servizio Google e la condivisione del foglio (procedura nella 5.8).

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi il Polso finanziario con lettura da Google Sheets. Autenticazione con account di servizio (variabili `GOOGLE_SHEETS_FINANCE_ID`, `GOOGLE_SERVICE_ACCOUNT_EMAIL`, `GOOGLE_SERVICE_ACCOUNT_KEY`), mai con la pubblicazione sul web. La rotta scarica il file, trasforma ogni foglio in una griglia di testo e manda tutto al modello con l'istruzione: estrai patrimonio netto, valuta, data di riferimento e categorie con nome, valore e tipo (liquidità, investito, debito); evita i doppi conteggi tra fogli di riepilogo e fogli di dettaglio; dalle tabelle con date usa solo la riga più recente; aggiungi un campo note per le ambiguità; rispondi solo in JSON. Valida il JSON e salva un'istantanea giornaliera nel log. La scheda legge sempre l'ultima istantanea e mostra patrimonio, delta e categorie; l'estrazione parte solo dal pulsante di aggiornamento o dal cron notturno, mai al caricamento della pagina. Accanto al pulsante, l'ora dell'ultima estrazione e il campo note se non è vuoto.

Verifica: premi il pulsante e confronta il patrimonio col foglio, a mano — le prime tre volte, non solo la prima. Togli la condivisione all'account di servizio e riprova: deve fallire con un errore leggibile,

non con un numero sbagliato. E ricaricando la pagina il consumo resta fermo: legge l'ultima istantanea, non il foglio.

A14 · La rotta delle domande L

Produce l'endpoint che risponde sulle tue cose passando tutto il contesto al modello. Se un giorno salirai al Completo, la adatterai con A16.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi la rotta delle domande. Raccogli profilo, memoria, catture recenti, obiettivi, persone e salute, passali tutti al modello insieme alla domanda, e chiedi di rispondere citando sempre da quale voce arriva ogni affermazione. Se un dato non c'è, deve dirlo invece di inventarlo.

Verifica: una domanda su un dato che c'è torna con la fonte citata; una domanda su un dato che non c'è torna con "non lo so", non con una risposta plausibile. Se torna vuota, Parte 8: "La risposta alle domande arriva vuota".

A15 · Lo strato di memoria C

Produce gli embedding a ogni scrittura e la rotta di ricerca per somiglianza. Serve l'estensione `vector` attiva (Parte 3).

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi lo strato di memoria. A ogni scrittura, calcola l'embedding del testo e salvalo. Il modello di embedding leggilo dalla variabile `OPENAI_EMBEDDING_MODEL`, lasciando `text-embedding-3-small` come predefinito nel codice. Aggiungi una rotta di ricerca che trasforma la domanda con lo stesso modello, chiama via `rpc()` la funzione `match_memoria` creata nella migrazione — l'operatore di distanza non si esprime con le query normali del client — e restituisce le venti voci più vicine con la riga di origine.

Verifica: cattura una cosa, poi cercala con parole diverse ma dal senso vicino: deve uscire tra i risultati.

A16 · Le domande sopra la memoria C

Produce la rotta delle domande che usa la ricerca di A15 invece del contesto pieno.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Crea la rotta delle domande — o adatta quella esistente, se arrivi dal Locale. Invece di passare tutto il contesto, usa la rotta di ricerca per prendere le venti voci più vicine alla domanda e passa al modello solo quelle, insieme al profilo. Le regole restano le stesse: citare sempre da dove arriva ogni affermazione, e se un dato non c'è dirlo invece di inventarlo.

Verifica: le stesse due prove di A14 — fonte citata quando il dato c'è, ammissione quando non c'è.

A17 · La barra che riconosce le domande L+C

Produce il doppio uso della barra di cattura: la stessa barra archivia le frasi e risponde alle domande.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Fai in modo che la barra riconosca le domande: se il testo finisce con un punto interrogativo o comincia con una parola interrogativa, invece di archiviare manda alla rotta delle domande e mostra la risposta espandendosi verso l'alto. Chiusura con la X e con Esc.

Verifica: "ricordati di chiamare Marco" viene archiviato; "cosa mi ero detto su Marco?" apre la risposta verso l'alto, e Esc la chiude.

A questo punto, sul Percorso Completo, si va online. Il deploy (7.2), le variabili d'ambiente (7.3) e la registrazione del webhook (7.4) sono comandi, non prompt: li trovi nella Parte 7, insieme alla lista di verifica finale (7.7). Poi restano gli ultimi due prompt.

A18 · Il briefing del mattino C

Produce l'endpoint del cron e la voce in `vercel.json`. È il prompt della Parte 7.5 — l'orario è in UTC, ricordalo due volte l'anno.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Costruisci l'endpoint del briefing del mattino in `/api/cron/briefing`.

1. Verifica l'intestazione `Authorization: Bearer` con `CRON_SECRET`; se non corrisponde rispondi 401 e non fare altro.
2. Controlla se il briefing di oggi è già stato inviato; se sì esci subito.
3. Raccogli: eventi di oggi, task con urgenza "oggi", obiettivi della settimana, cosa risulta slittato da ieri.
4. Una sola chiamata a Claude che trasformi questi dati in un messaggio breve, in italiano, massimo 10 righe, senza convenevoli.
5. Invia su Telegram con `sendMessage` a `TELEGRAM_USER_ID`.
6. Registra l'invio nel registro, così una seconda esecuzione non manda un doppione.
7. Rispondi sempre 200 con un riepilogo di cosa è stato fatto.

Aggiungi la voce in `vercel.json` con `schedule "0 5 * * *"` e scrivi nel codice un commento che ricordi che l'orario è UTC.

Verifica: chiama l'endpoint a mano con l'intestazione `Bearer` (comando nella Parte 7.5): il briefing arriva su Telegram. Chiamalo una seconda volta: non arriva un doppione.

A19 · L'export di backup

Produce l'endpoint di export completo e il pulsante per scaricarlo. È il prompt della Parte 7.6 — ed è anche la tua via d'uscita da qualsiasi servizio.

PROMPT · DA INCOLLARE IN CLAUDE CODE

Aggiungi un endpoint `/api/admin/export` che:

- accetta solo richieste che portano un cookie di sessione valido: qui l'intestazione `x-api-secret` **non** vale, e va rifiutata esplicitamente — è la rotta che restituisce tutto il database, e la porta di servizio lì non entra
- legge tutte le tabelle e restituisce un JSON unico con dentro tutto, più la data di generazione e la versione dello schema
- imposta le intestazioni per farlo scaricare come file

Aggiungi un pulsante "Scarica backup" nelle impostazioni della dashboard che chiama questo endpoint.

Verifica: scarica il file e aprilo: dentro ci sono tutte le tabelle, la data di generazione e la versione dello schema. Se una tabella manca, te ne accorgi adesso — non il giorno che il backup ti serve.

A20 • Il travaso dal Locale al Completo [L → C](#)

Produce lo script una-tantum che porta i dati del file JSON dentro le tabelle, embedding compresi. Riguarda solo chi arriva dal Locale, e si fa **dopo** A6 — tabelle create e 🖐️ migrazione applicata — dopo A7, che porta lo store sul database, e dopo A15, che è la rotta che genera gli embedding. È la voce 7 della Parte 9.

PROMPT • DA INCOLLARE IN CLAUDE CODE

Ho una dashboard PersonalOS sul Percorso Locale, con i dati in `data/personalos.json`, e ho appena creato le tabelle su Supabase. Scrivi uno script una-tantum che legge quel file e travasa nelle tabelle: profilo, task, persone, catture, log giornalieri. Rispetta gli identificativi esistenti dove ci sono, e riporta a schermo quante righe ha scritto per ogni tabella. Poi rigenera gli embedding di tutte le voci di memoria con la rotta dello strato di memoria, in blocchi, mostrando l'avanzamento. Il file locale non si cancella e non si modifica: resta lì come backup.

Verifica: i conteggi per tabella che lo script ti stampa coincidono con quelli che leggi nel file. Poi una domanda alla memoria su una cosa vecchia: se torna con la fonte citata, gli embedding ci sono tutti.

Appendice B · Checklist variabili d'ambiente

Tutte le variabili che il sistema usa, divise per percorso.

Come si leggono i percorsi: L Locale · C Completo · L+C entrambi

In locale stanno in un file `.env.local` alla radice del progetto, **mai nel controllo di versione** (controlla che `.gitignore` contenga `.env*`). Online si impostano dal pannello del servizio o dalla riga di comando, **per tutti gli ambienti**, non solo produzione. Dopo ogni modifica serve un nuovo deploy.

Modelli

Variabile	A cosa serve	Dove si ottiene	Percorso
<code>ANTHROPIC_API_KEY</code>	Smistamento delle catture, risposte sulla memoria, tutte le sintesi	<code>console.anthropic.com</code> → API Keys. Serve credito sull'account, è separato dall'abbonamento a Claude	L+C
<code>ANTHROPIC_MODEL</code>	Il nome del modello. Nel codice resta solo un valore predefinito: la variabile serve a cambiarlo senza toccare i file	Lo scegli tu dalla documentazione Anthropic. Aggiornalo quando cambi modello	L+C
<code>OPENAI_API_KEY</code>	Trascrizione delle note vocali (Whisper) e generazione degli embedding	<code>platform.openai.com</code> → API Keys. Anche qui serve credito	C
<code>OPENAI_EMBEDDING_MODEL</code>	Il modello di embedding: lo legge la rotta di ricerca della memoria, che nel codice tiene <code>text-embedding-3-small</code> come predefinito	Lo scegli tu. Se lo cambi devi rigenerare tutti gli embedding — vettori fatti con modelli diversi non sono confrontabili — e adeguare la dimensione della colonna <code>vector(1536)</code> della migrazione, perché la lunghezza del vettore cambia col modello	C

⚠ **Attenzione al costo.** Queste due chiavi sono le uniche voci che consumano davvero. Metti un limite di spesa mensile sul pannello di entrambi i fornitori il giorno stesso in cui crei la chiave, prima ancora di scrivere codice. Non è pessimismo: è che un ciclo sbagliato che chiama il modello a ogni render lo scopri dalla bolletta.

Database

Variabile	A cosa serve	Dove si ottiene	Percorso
NEXT_PUBLIC_SUPABASE_URL	L'indirizzo del tuo progetto database	Pannello Supabase → Project Settings → API	
NEXT_PUBLIC_SUPABASE_ANON_KEY	Chiave pubblica, quella che può stare nel browser: da sola non apre niente perché le regole di accesso la fermano. Il codice di questa guida non la usa — tutto passa dal server con la chiave di servizio — ma serve alla verifica delle regole di riga (A6, A7) e a qualsiasi estensione lato client: tienila	Stesso posto	
SUPABASE_SERVICE_ROLE_KEY	Chiave amministrativa: scavalca tutte le regole di accesso . La usano solo le rotte sul server	Stesso posto	

⚠ **La riga più pericolosa di tutta l'appendice.** La chiave di servizio non deve **mai** avere il prefisso NEXT_PUBLIC_ . Quel prefisso non è un nome: è un'istruzione che dice al framework di inserire il valore nel codice che scarica il browser. Con quella chiave in mano chiunque legge e scrive tutto il tuo database, regole di accesso comprese. Se per errore ci finisce anche solo una volta, rigenerala dal pannello: considerala compromessa.

Calendario

Variabile	A cosa serve	Dove si ottiene	Percorso
GOOGLE_CALENDAR_ICAL_URL	L'indirizzo segreto del calendario in formato iCal, da cui la scheda Calendario legge gli impegni	Google Calendar → impostazioni del calendario → "Indirizzo segreto in formato iCal"	L+C

⚠ **Quell'indirizzo è una password.** Chiunque lo conosca legge tutto il tuo calendario. Mai nel codice, mai in un commit, mai in uno screenshot; se ti scappa, rigeneralo dalla stessa pagina delle impostazioni e quello vecchio muore.

Finanze

Variabile	A cosa serve	Dove si ottiene	Percorso
FINANCE_FILE_PATH	Il percorso del file del foglio di calcolo (xlsx o csv) da cui il Polso finanziario estrae i numeri	Lo decidi tu: è dove tieni l'export del tuo foglio	L
GOOGLE_SHEETS_FINANCE_ID	L'identificativo del foglio Google delle finanze	Sta nell'indirizzo del foglio: è il blocco tra <code>/d/</code> e <code>/edit</code>	C
GOOGLE_SERVICE_ACCOUNT_EMAIL	L'email dell'account di servizio che legge il foglio	Console Google Cloud → account di servizio (procedura nella Parte 5.8). Finisce in <code>iam.gserviceaccount.com</code>	C
GOOGLE_SERVICE_ACCOUNT_KEY	La chiave privata dell'account di servizio	Dal file JSON generato per l'account	C

⚠ **Due cose su questo blocco.** Il foglio si condivide **solo** con l'email dell'account di servizio, come Visualizzatore — mai con la pubblicazione sul web, che genera un indirizzo pubblico. E la chiave privata contiene degli a-capo: incollala con attenzione, perché gli a-capo mangiati sono la causa numero uno di "chiave non valida" con la chiave giusta (Parte 7.3).

Telegram

Variabile	A cosa serve	Dove si ottiene	Percorso
TELEGRAM_BOT_TOKEN	Identifica il bot: serve per ricevere i messaggi e per rispondere	Da @BotFather su Telegram: <code>/newbot</code> , scegli nome e username. Te lo dà una volta sola — se lo perdi lo rigeneri	
TELEGRAM_WEBHOOK_SECRET	Telegram lo rimanda in un'intestazione a ogni chiamata: è come il tuo endpoint distingue Telegram da chiunque altro	Te lo generi: <code>openssl rand -hex 32</code> . Va passato al comando <code>setWebhook</code>	
TELEGRAM_USER_ID	Il tuo ID numerico. Il bot ignora chiunque altro, ed è anche l'indirizzo a cui il briefing viene inviato	Scrivi a @userinfobot su Telegram: risponde con il tuo ID	

⚠ Un bot Telegram è pubblico: chiunque conosca l'username può scrivergli. Il controllo sull'ID utente è ciò che impedisce a un estraneo di scrivere nel tuo sistema. Non è opzionale.

Sicurezza

Variabile	A cosa serve	Dove si ottiene	Percorso
<code>AUTH_SECRET</code>	Firma i cookie di sessione: è quello che rende impossibile fabbricarsene uno valido	<code>openssl rand -hex 32</code> . Incollalo senza guardarlo. Se lo cambi, tutte le sessioni aperte cadono	C
<code>DASHBOARD_PASSWORD</code>	La password che digiti tu per entrare	La scegli tu. Una frase lunga , non otto caratteri: niente impedisce a qualcuno di provarne diecimila	C
<code>API_SECRET</code>	Accesso programmatico alle API senza passare dal login: scorciatoie del telefono, script. Sulla rotta di export non vale: quella vuole il cookie di sessione (A19)	<code>openssl rand -hex 32</code>	C
<code>CRON_SECRET</code>	Autentica le chiamate pianificate. Senza, gli endpoint del cron sono pulsanti pubblici che consumano il tuo budget	<code>openssl rand -hex 32</code> . Su Vercel viene inviato in automatico come <code>Authorization: Bearer</code>	C

Varie

Variabile	A cosa serve	Dove si ottiene	Percorso
<code>NEXT_PUBLIC_APP_URL</code>	Il dominio del deploy: è l'indirizzo pubblico del sistema, e lo usi per registrare il webhook (7.4), per chiamare il cron a mano (7.5) e per i link dentro i messaggi	Te lo dà il servizio di hosting dopo il primo deploy	C
<code>USER_TIMEZONE</code>	Il fuso in cui vivi, per esempio <code>Europe/Rome</code> . Decide quando "oggi" diventa "ieri"	Lo imposti tu	L+C
<code>USER_ID</code>	Identificativo fisso dell'unico utente: valorizza la colonna <code>user_id</code> che A6 mette su ogni tabella, così le query nascono filtrate e le tabelle sono già pronte al giorno in cui gli utenti diventano due	Lo scegli tu, una stringa qualsiasi	C

⚠ **Sul fuso orario.** Se non lo dichiari, il server userà il suo — che è UTC. Il risultato è che le abitudini si azzerano all'una o alle due di notte mentre sei ancora sveglio, e il conteggio del giorno è sbagliato senza che niente segnali un errore. Usa una funzione sola per rispondere alla domanda "che giorno è oggi?" e chiamala ovunque.

I due file, per intero

Percorso Locale — `.env.local`. Si parte con una riga:

```
ANTHROPIC_API_KEY=sk-ant-...
```

E fino alla Parte 5 basta quella. Le altre arrivano quando costruisci le schede che le usano — al completo sono queste:

```
ANTHROPIC_API_KEY=sk-ant-...
ANTHROPIC_MODEL=
USER_TIMEZONE=Europe/Rome
GOOGLE_CALENDAR_ICAL_URL=
FINANCE_FILE_PATH=
```

`ANTHROPIC_MODEL` e `USER_TIMEZONE` puoi anche ometterle: il codice ha i suoi predefiniti. Le ultime due servono al Calendario (5.2) e al Polso finanziario (5.8).

Percorso Completo — quello che finisce nel pannello dell'hosting:

```
ANTHROPIC_API_KEY=
ANTHROPIC_MODEL=
OPENAI_API_KEY=
OPENAI_EMBEDDING_MODEL=

NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

GOOGLE_CALENDAR_ICAL_URL=
GOOGLE_SHEETS_FINANCE_ID=
GOOGLE_SERVICE_ACCOUNT_EMAIL=
GOOGLE_SERVICE_ACCOUNT_KEY=

TELEGRAM_BOT_TOKEN=
TELEGRAM_WEBHOOK_SECRET=
TELEGRAM_USER_ID=

AUTH_SECRET=
DASHBOARD_PASSWORD=
API_SECRET=
CRON_SECRET=

NEXT_PUBLIC_APP_URL=
USER_TIMEZONE=Europe/Rome
USER_ID=
```

Ultima verifica, e falla davvero. Prendi questo elenco e confrontalo con quello che il codice legge realmente:

```
grep -rho "process\.env\. [A-Z_]*" --exclude-dir=node_modules --include="*.js" --inc
```

Se il codice legge una variabile che qui non c'è, l'hai dimenticata e te ne accorgerai in produzione. Se qui c'è una variabile che il codice non legge, o è avanzata da una funzione che hai tolto, o — più probabile — c'è un pezzo che non hai ancora costruito.

Prima di chiudere

Adesso costruiscilo. Poi usalo per una settimana intera senza cambiargli niente: è l'unico modo di scoprire quali pezzi ti servono davvero e quali hai messo lì perché stavano bene. Dopo quella settimana, piegalo alle tue abitudini — non il contrario.

Se costruisci qualcosa con questa guida, fammelo vedere: taggami su Instagram — [instagram.com/giu.castagna](https://www.instagram.com/giu.castagna).

Il resto lo trovi sul canale: [youtube.com/@gcastagna](https://www.youtube.com/@gcastagna).